

MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing

Nika Mansouri Ghiasi¹ Mohammad Sadrosadati¹ Harun Mustafa¹ Arvid Gollwitzer¹
 Can Firtina¹ Julien Eudine¹ Haiyu Mao¹ Joël Lindegger¹ Meryem Banu Cavlak¹
 Mohammed Alser¹ Jisung Park² Onur Mutlu¹
¹ETH Zürich ²POSTECH

Metagenomics, the study of the genome sequences of diverse organisms in a common environment, has led to significant advances in many fields. Since the species present in a metagenomic sample are not known in advance, metagenomic analysis commonly involves the key tasks of determining the species present in a sample and their relative abundances. These tasks require searching large metagenomic databases containing information on different species' genomes. Metagenomic analysis suffers from significant data movement overhead due to moving large amounts of low-reuse data from the storage system to the rest of the system. In-storage processing can be a fundamental solution for reducing this overhead. However, designing an in-storage processing system for metagenomics is challenging because existing approaches to metagenomic analysis cannot be directly implemented in storage effectively due to the hardware limitations of modern SSDs.

We propose MegIS, the first in-storage processing system designed to significantly reduce the data movement overhead of the end-to-end metagenomic analysis pipeline. MegIS is enabled by our lightweight design that effectively leverages and orchestrates processing inside and outside the storage system. Through our detailed analysis of the end-to-end metagenomic analysis pipeline and careful hardware/software co-design, we address in-storage processing challenges for metagenomics via specialized and efficient 1) task partitioning, 2) data/computation flow coordination, 3) storage technology-aware algorithmic optimizations, 4) data mapping, and 5) lightweight in-storage accelerators. MegIS's design is flexible, capable of supporting different types of metagenomic input datasets, and can be integrated into various metagenomic analysis pipelines. Our evaluation shows that MegIS outperforms the state-of-the-art performance- and accuracy-optimized software metagenomic tools by $2.7\times$ – $37.2\times$ and $6.9\times$ – $100.2\times$, respectively, while matching the accuracy of the accuracy-optimized tool. MegIS achieves $1.5\times$ – $5.1\times$ speedup compared to the state-of-the-art metagenomic hardware-accelerated (using processing-in-memory) tool, while achieving significantly higher accuracy.

1. Introduction

Metagenomics, an increasingly important domain in bioinformatics, requires the analysis of the genome sequences of various organisms of *different species* present in a common environment (e.g., human gut, soil, or oceans) [1–3]. Unlike traditional genomics [4–8] that studies genome sequences from an individual (or a small group of individuals) of the *same known species*, metagenomics deals with genome sequences whose species are *not known* in advance in many cases, thereby requiring comparisons of the target sequences against large databases of many reference genomes. Metagenomics has led to groundbreaking advances in many fields, such as precision

medicine [9,10], urgent clinical settings [11], understanding microbial diversity of an environment [12,13], discovering early warnings of communicable diseases [14–16], and outbreak tracing [17]. The pivotal role of metagenomics, together with rapid improvements in genome sequencing (e.g., reduced cost and improved throughput [18]), has resulted in the fast-growing adoption of metagenomics [10,19,20].

Given a metagenomic sample, a typical workflow consists of three key steps: (i) sequencing, (ii) basecalling, and (iii) metagenomic analysis. First, *sequencing* extracts the genomic information of *all* organisms in the sample. Since current sequencing technologies *cannot* process a DNA molecule as a whole, a sequencing machine generates randomly sampled, inexact fragments of genomic information, called *reads*. A metagenomic sample contains organisms from several species, and during sequencing, it is unclear what species each read comes from. Second, *basecalling* converts the raw sequencer data of reads into sequences of characters that represent the nucleotides A, C, G, and T. Third, *metagenomic analysis* identifies the distribution of different taxa (i.e., groups or categories in biological classification, such as species and genera) within a metagenomic sample. Metagenomic analysis commonly involves the two key tasks of determining the species *present/absent* in the sample and finding their *relative abundances*.

To enable fast and efficient metagenomics for many critical applications, it is essential to improve the performance and energy efficiency of metagenomic analysis due to at least three major reasons. First, metagenomic analysis is typically performed much more frequently compared to the other two steps (i.e., sequencing and basecalling) in the metagenomic workflow. While sequencing and basecalling are one-time tasks for a sample in many cases, sequenced and basecalled reads in a sample often need to be analyzed over and over in *multiple studies* or at *different times* in the same study [21]. Second, as shown in our motivational analysis in §3 on a high-end server node, even when performing the metagenomic analysis step only once for a sample, this step bottlenecks the end-to-end performance and energy efficiency of the workflow. Third, the performance and energy-efficiency gaps between the metagenomic analysis step and the other steps are expected to widen even more due to the rapid advances in sequencing and basecalling technologies, such as significant increases in throughput and energy efficiency of sequencing [22–27] and basecalling [28–33]. For these reasons, simply scaling up traditional systems *cannot* effectively optimize the metagenomic analysis step enough to keep up with the rapid advances.

Metagenomic analysis suffers from significant data movement overhead because it requires accessing large amounts of low-reuse data. Since we do not know the species present in a metagenomic sample, metagenomic analysis requires search-

ing large databases (e.g., to several TBs [34–39] or more than a hundred TBs in emerging databases [36, 37]) that contain information on different organisms’ genomes. Database sizes are expected to increase further in the future, and at a fast pace.¹ Two notable reasons for this growth are 1) the rapid evolution of viruses and bacteria [41], which necessitates frequent updates with new reference genomes [42, 43], and 2) the fact that databases may include sequences from both highly curated reference genomes and from less curated metagenomic sample sets [35, 36]. Particularly, as over 99% of Earth’s microbes remain unidentified and excluded from curated reference genome databases [44, 45], the expanded databases improve sensitivity [45]. Recent advances in the automated and scalable construction of genomic data from more organisms have further contributed to database growth by enabling the rapid addition of new sequences to databases [46, 47]. Our motivational analysis (§3) of the state-of-the-art metagenomic analysis tools shows that data movement overhead from the storage system significantly impacts their end-to-end performance. Due to its low reuse, the data needs to move all the way from the storage system to the main memory and processing units for its first use, and it will likely not be used again or reused very little during analysis. This unnecessary data movement, combined with the low computation intensity of metagenomic analysis and the limited I/O (input/output) bandwidth, leads to large storage I/O overheads for metagenomic analysis.

While there has been effort in accelerating metagenomic analysis, to our knowledge, no prior work fundamentally addresses its storage I/O overheads. Some works (e.g., [48–53]) aim to alleviate this overhead by applying sampling techniques to reduce the database size, but they incur accuracy loss, which is problematic for many use cases (e.g., [18, 26, 42, 54–60]). Various other works (e.g., [61–70]) accelerate other bottlenecks in metagenomic analysis, such as computation and main memory bottlenecks. These works do not alleviate I/O overheads, whose impact on end-to-end performance becomes even larger (as shown in §3) when other bottlenecks are alleviated.

In-Storage Processing (ISP), i.e., processing data directly inside the storage device where target data resides, can be a fundamentally high-performance approach to mitigating the data-movement bottleneck in metagenomic analysis, given its three major benefits. First, ISP can significantly reduce unnecessary data movement from/to the storage system by processing large amounts of low-reuse data inside the storage system while sending only the results to the host. Second, ISP can leverage each SSD’s² large internal bandwidth to access target data without being restricted by the SSD’s relatively smaller external bandwidth. Third, ISP alleviates the overall execution burden of applications with low data reuse from the rest of the system (e.g., processing units and main memory), freeing up the host to perform other useful work instead.

Challenges of ISP. Despite the benefits of ISP, none of the existing approaches to metagenomic analysis can be effectively implemented as an ISP system due to the limited hardware resources available in current storage devices. Some tools incur

a large number of random accesses to search the database (e.g., [48, 49, 51, 53, 54, 64, 73–78]), which hinders ISP’s large potential by preventing the full utilization of SSD internal bandwidth. This is due to costly conflicts in internal SSD resources (e.g., channels and NAND flash chips [79–81]) caused by random accesses. Some tools predominantly incur more suitable streaming accesses (e.g., [38, 52, 57, 82, 83]), but doing so comes at the cost of more computation and main memory capacity requirements that are challenging for ISP to meet due to the limited hardware resources available inside SSDs. Therefore, directly adopting either approach (with random or streaming accesses) in ISP incurs performance, energy, and storage device lifetime overheads.

Our goal in this work is to improve metagenomic analysis performance by reducing the large data movement overhead from the storage system in a cost-effective manner. To this end, we propose **MegIS**, the *first* ISP system designed to reduce the data movement overheads inside the end-to-end metagenomic analysis pipeline. The key idea of MegIS is to enable *cooperative* ISP for metagenomics, where we do not solely focus on processing inside the storage system but, instead, we capitalize on the strengths of processing *both inside and outside* the storage system. We enable cooperative ISP via a synergistic hardware/software co-design between the storage system and the host system.

Key Mechanism. We design MegIS as an efficient pipeline between the SSD and the host system to (i) *leverage* and (ii) *orchestrate* the capabilities of both. Based on our rigorous analysis of the end-to-end metagenomic analysis pipeline, we propose a new hardware/software co-designed accelerator framework that consists of five aspects. First, we partition and map different parts of the metagenomic analysis pipeline to the host and the ISP system such that each part is executed on the most suitable architecture. Second, we coordinate the data/computation flow between the host and the SSD such that MegIS (i) completely overlaps the data transfer time between them with computation time to reduce the communication overhead between different parts, (ii) leverages SSD bandwidth efficiently, and (iii) does not require large DRAM inside the SSD or a large number of writes to the flash chips. Third, we devise storage technology-aware metagenomics algorithm optimizations to enable efficient access patterns to the SSD. Fourth, we design lightweight in-storage accelerators to perform MegIS’s ISP functionalities while minimizing the required SRAM/DRAM buffer spaces inside the SSD. Fifth, we design an efficient data mapping scheme and Flash Translation Layer (FTL) specialized to the characteristics of metagenomic analysis to leverage the SSD’s full internal bandwidth.

Key Results. We evaluate MegIS with two different SSD configurations (performance-optimized [84] and cost-optimized [85]). We compare MegIS against three state-of-the-art software and hardware-accelerated metagenomics tools: (i) Kraken2 [49], which is optimized for performance, (ii) Metalign [82], which is optimized for accuracy, and (iii) a state-of-the-art processing-in-memory accelerator, Sieve [64], integrated into Kraken2 to accelerate its k-mer matching. By analyzing *end-to-end* performance, we show that MegIS provides $2.7\text{--}37.2\times$ and $1.5\text{--}5.1\times$ speedup compared to Kraken2 and Sieve, respectively, while achieving significantly higher accuracy. MegIS provides $6.9\text{--}100.2\times$ speedup compared to

¹For example, based on recently published trends, the ENA assembled/annotated sequence database size currently *doubles* every 19.9 months [25], and the BLAST nt database size doubled from 2021 to 2022 [40].

²In this work, we focus on the predominant NAND flash-based SSD technology [71, 72]. We expect that our insights and designs would benefit storage systems built with other emerging technologies.

Metalign, while providing the *same* accuracy (MegIS does not affect analysis accuracy compared to this accuracy-optimized baseline). MegIS provides large average energy reductions of $5.4\times$ and $1.9\times$ compared to Kraken2 and Sieve, respectively, and $15.2\times$ compared to accuracy-optimized Metalign. MegIS's benefits come at a low area cost of 1.7% over the area of the three cores [86] in an SSD controller [87].

This work makes the following **key contributions**:

- We demonstrate the end-to-end performance impact of storage I/O overheads in metagenomic analysis.
- We propose **MegIS**, the *first* in-storage processing (ISP) system tailored to reduce the data movement overhead of the end-to-end metagenomic analysis pipeline, significantly reducing its I/O overheads and improving its performance and energy-efficiency.
- We present a new hardware/software co-design to enable an efficient and cooperative pipeline between the host and the SSD to alleviate I/O data movement overheads in metagenomic analysis.
- We rigorously evaluate MegIS and show that it improves performance and energy efficiency compared to the state-of-the-art metagenomics tools (software and hardware-accelerated), while maintaining high accuracy. It does so without relying on costly hardware resources throughout the system, making metagenomics more accessible for wider adoption.

2. Background

2.1. Metagenomic Analysis

Fig. 1 shows an overview of metagenomic analysis, which involves determining the species *present/absent* in the sample ① and their *relative abundances* (i.e., the relative frequencies of the occurrence of different species in the sample) ②.

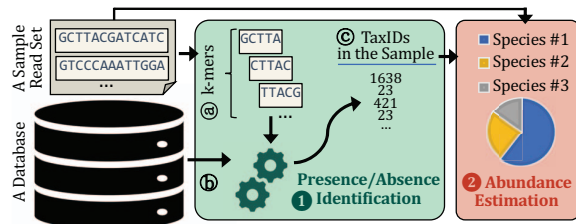


Figure 1: Overview of metagenomic analysis.

2.1.1. Presence/absence Identification. To find species present in the sample, many tools (e.g., [38, 48, 49, 51–54, 57, 64, 73–78, 82, 83]) extract *k-mers* (i.e., subsequences of length k) from the input queries in a sample read set (a in Fig. 1) and search for the *k-mers* in an input *reference database* (b). Each database contains *k-mers* extracted from reference genomes of a wide range of species. The database associates each indexed *k-mer* with a *taxonomic identifier (taxID)*³ of the reference genome(s) the *k-mer* comes from. At the end of the presence/absence identification process, the metagenomic tool outputs the *taxIDs* of the species present in the sample (c).

The GB- or TB-scale databases typically support random (e.g., [48, 49, 51, 53, 54, 64, 73–78]) or streaming (e.g., [38, 52, 57, 82, 83]) access patterns.

Tools with Random Access Queries (R-Qry). Some tools (e.g., [48, 49, 51, 53, 54, 64, 73–78]) commonly perform random accesses to search their database. A state-of-the-art tool in

this category is Kraken2 [49], which maintains a hash table that maps each indexed *k-mer* to a *taxID*. To identify which species are present in a set of queries, Kraken2 extracts *k-mers* from the read queries and searches the hash table to retrieve the *k-mers*' associated *taxIDs*. For each read, Kraken2 collects the *taxIDs* of that read's *k-mers* and, based on the occurrence frequencies of these *taxIDs*, uses a classification algorithm to assign a single *taxID* to each read. Finally, Kraken2 identifies the species present in the sample based on the *taxIDs* of the reads in the sample.

Tools with Streaming Access Queries (S-Qry). Some tools (e.g., [38, 52, 57, 82, 83]) predominantly feature streaming accesses to their databases. A state-of-the-art tool in this category is Metalign [82]. Presence/absence identification in Metalign is done via 1) preparing the input read set queries, and 2) finding species present in them. To process the queries, the tool extracts *k-mers* from the reads and sorts them. Finding the species in the sample involves two steps. First, the tool finds the *intersecting k-mers*, which are *k-mers* that are common between the query *k-mers* and a pre-sorted reference database. In this step, the tool uses large *k-mers* (e.g., $k = 60$) for both the queries and the database to maintain a low false positive rate. This is because large *k-mers* are more unique, and matching a long *k-mer* ensures that the queries have at least one long and specific match to the database. Second, the tool finds the *taxIDs* of the intersecting *k-mers* by searching for the intersecting *k-mers* or their prefixes in a smaller *sketch database* of variable-sized *k-mers*. Each *sketch* is a small representative subset of *k-mers* associated with a given *taxID*. Searching for both the intersecting *k-mers* and their prefixes in this step increases the true positive rate (i.e., species correctly identified as present in the sample out of all species actually present in the sample) by expanding the number of matches.

2.1.2. Abundance Estimation. After finding the *taxIDs* of the species present in the sample, some applications require a more sensitive step to find the species' relative abundances [48, 51, 53, 74, 76, 82, 83, 88–91] in the sample. Different tools implement their own approaches for estimating abundances, from lightweight statistical models [89–91] to more accurate but computationally-intensive read mapping [48, 53, 54, 74, 76, 82]. Read mapping is the process of finding potential matching locations of reads against one or more reference genomes. Metagenomic tools can map the reads against reference genomes of species in the sample, accurately determining the number of reads belonging to each species.

2.2. SSD Organization

Fig. 2 depicts the organization of a modern NAND flash-based SSD, which consists of three main components.

① NAND Flash Memory. A NAND package consists of multiple *dies* or *chips*, sharing the NAND package's I/O pins. One or multiple packages share a command/data bus or *channel* to communicate with the SSD controller. Dies can operate

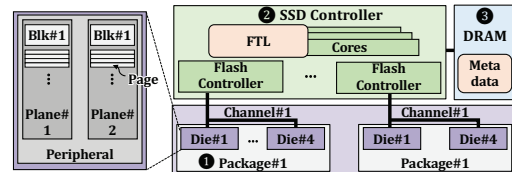


Figure 2: Organizational overview of a modern SSD.

³A *taxID* is an integer attributed to a cluster of related species.

independently, but each channel can be used by only one die at a time to communicate with the controller. Each die has multiple (e.g., 2 or 4) *planes*, each with thousands of *blocks*. Each block has hundreds to thousands of 4–16 KiB *pages*. NAND flash memory performs read/write operations at page granularity but erase operations at block granularity. The peripheral circuitry to access pages is shared among the planes in each die. Hence, it is possible for the planes in a die to operate concurrently when accessing pages (or blocks) at the same offset. This mode is called the *multiplane* operation.

② **SSD Controller.** An SSD controller consists of two key components. First, multiple cores run the FTL, which is responsible for communication with the host, internal I/O scheduling, and various SSD management tasks. Second, per-channel hardware flash controllers manage request handling [80, 92] and error correction for the NAND flash chips [71, 72, 93–104].

③ **DRAM.** Modern SSDs use low-power DRAM [105] to store metadata for SSD management tasks. Most of the DRAM capacity inside the SSD (i.e., internal DRAM) is used to store the logical-to-physical (i.e., L2P) mappings, which are typically maintained at a granularity of 4KiB to enhance random access performance. In a 32-bit architecture, with 4 bytes of metadata stored for every 4KiB of data, the required capacity for L2P mappings is about 0.1% of the SSD’s capacity. For example, a 4-GB LPDDR4 DRAM is used for a 4-TB SSD [87].

2.3. In-Storage Processing

Processing data directly in storage via *in-storage processing* (ISP) can be a fundamentally high-performance approach for reducing the overheads of moving large amounts of low-reuse data across the system, providing three key benefits. First, ISP reduces unnecessary data movement from the storage system. Second, ISP reduces the execution burden of applications with low data reuse from the rest of the system, allowing it to perform other useful tasks. Third, as shown by many prior works (e.g., [106–114]), ISP can benefit from the SSD’s internal bandwidth. In modern SSDs, the internal bandwidth is usually larger than the external. For example, a modern SSD controller [115] supports 6.5 GB/s external bandwidth and 19.2 GB/s internal bandwidth (16 channels with a maximum per-channel bandwidth of 1.2 GB/s). It is essential to overprovision the internal bandwidth to avoid hurting the user-perceived external I/O bandwidth by reducing the negative impact of 1) channel conflicts [79, 81, 116, 117] and 2) the SSD’s internal data migration for management tasks such as garbage collection [71, 80, 118–120], wear-leveling [71, 72, 121], and data refresh [71, 98, 101, 102].

Some prior works propose ISP systems in the form of *special-purpose* accelerators for different applications [106, 107, 111, 112, 114, 122–142]. Several prior works propose *general-purpose* processing inside storage devices [105, 138, 139, 143–154], bitwise operations using NAND flash [155, 156], or SSDs in close integration with FPGAs [109, 157–161] or GPUs [162].

3. Motivational Analysis

3.1. Criticality of Metagenomic Analysis

By enabling the analysis of the genomes of organisms from different species in a common environment, metagenomics overcomes a limitation of traditional genomics, which requires culturing individual known species in isolation. This limitation has been a major roadblock in many clinical and environmental

use cases [163]. The impact of metagenomics has been rapidly increasing in many areas that each have broad implications for society, such as health [9, 11], agriculture [164], environmental monitoring [14–17], and many other critical areas. Due to its importance, metagenomics has attracted wide global attention, with medical and government health institutions heavily investing in metagenomic analysis [164–166]. The global amount of genomic data that is incorporated in metagenomic workflows is growing exponentially [42, 167], doubling every several months [25, 40, 42, 168], and is projected to surpass the data growth rate of YouTube and Twitter [64, 167, 168].

In metagenomics, the analysis step bottlenecks the end-to-end performance of the workflow, and therefore, poses a pressing need for acceleration [11, 19, 64, 65, 169, 170] for three reasons. First, the sequencing and basecalling steps for a sample are usually one-time tasks [22, 26, 171]. In many cases, the reads from a single sequenced sample can be analyzed by *multiple studies* or *at different times* in the same study. This is because (i) there are many heuristics involved in metagenomics, and achieving a desired sensitivity-specificity tradeoff commonly requires parameter tuning [21], or using different databases created with different parameters or genomes [172], and (ii) a sample can be analyzed several times with databases that are regularly updated with new genomes, or with syndrome-specific targeted databases [172]. Second, even when performing the metagenomic analysis step only once for a sample, the *throughput* of this step is significantly lower than the sequencing throughput of modern sequencers (e.g., [173]). While sequencing one sample can take a long time, a single sequencing machine can sequence *many samples* from different sources in parallel [22, 174], achieving very high throughput. Our analysis with a state-of-the-art metagenomic tool [82] shows that analyzing the data, sequenced and basecalled by a sequencer in 48 hours, takes 38 days on a high-end server node (detailed configurations in §5). Such long analysis poses serious challenges, specifically for time-critical use cases (e.g., clinical settings [11] and timely surveillance of infectious diseases [17]). Since the growth rate of sequencing throughput is higher than Moore’s Law [18], this already large gap between sequencing and analysis throughput is widening [22, 24, 25, 27], and simply scaling up traditional systems for analysis is not efficient. Third, the development of sequencing technologies that enable analysis *during* sequencing [175–181] increasingly necessitates the need for fast analysis that can keep up with sequencing throughput.

The analysis step is also the primary energy bottleneck in the metagenomic workflow, and optimizing its efficiency is vital as sequencing technologies rapidly evolve. For example, a high-end sequencer [173] uses 405 KJ to sequence and basecall 100 million reads, with 92.5 Mbp/s throughput and 2,500 W power consumption [173]. In contrast, processing this dataset on a commodity server (detailed configurations in §5) requires 675 KJ, accounting for 63% of total energy. The need to enhance the analysis’ energy efficiency is further increasing for two reasons. First, sequencing efficiency has been continually improving. For example, a new version of Illumina sequencer from 2023 [173] provides 44× higher throughput at only 1.5× higher power consumption compared to an older version [182] from 2020, resulting in much better sequencing energy efficiency. Therefore, simply relying on scaling up commodity systems to improve the analysis throughput worsens the en-

ergy bottleneck. Second, the increased adoption of compact *portable sequencers* [183] for on-site metagenomics (e.g., in remote locations [184] or for personalized bedside care [19]) offers high-throughput sequencing with low energy costs. This further amplifies the need for energy- and cost-effective analysis that can match the portability and convenience of these sequencers.

3.2. Data Movement Overheads

We conduct experimental analysis to assess the storage system's impact on the performance of metagenomic analysis.

Tools and Datasets. We analyze two state-of-the-art tools for presence/absence identification: 1) Kraken2 [49], which queries its large database with random access patterns (R-Qry),⁴ and 2) Metalign [82], which exhibits mostly sequential streaming accesses to its database (S-Qry). We use the best-performing thread count for each tool. We use a query sample with 100 million reads (CAMI-L, detailed in §5) from the CAMI dataset [59], commonly used for profiling metagenomic tools. We generate a database based on microbial genomes drawn from NCBI's databases [82, 185] using default parameters for each tool. For Kraken2 [49], this results in a 293 GB database. For Metalign [82], this results in a 701 GB k-mer database and a 6.9 GB sketch tree. To show the impact of database size, we also analyze larger k-mer databases (0.6 TB and 1.4 TB for Kraken and Metalign, respectively) that include more species.

System Configurations. We use a high-end server with an AMD EPYC 7742 CPU [186] and 1.5-TB DDR4 DRAM [187]. We note that the DRAM size is larger than the size of all data accessed during the analysis by each tool. This way, we can analyze the fundamental I/O overhead of moving large amounts of low-reuse data from storage to the main memory without being limited by DRAM capacity. We evaluate I/O overheads using: 1) a cost-optimized SSD (SSD-C) [85] with a SATA3 interface [188], 2) a performance-optimized SSD (SSD-P) [84] with a PCIe Gen4 interface [189], and 3) a hypothetical configuration with zero performance overhead due to storage I/O (No-I/O). SSD-P provides an order-of-magnitude higher sequential-read bandwidth than SSD-C (detailed configurations in Table 1). However, scaling up storage capacity only using performance-optimized SSDs is challenging due to their much higher prices (e.g., [190–192]) and fewer PCIe slots compared to SATA slots available on servers (e.g., [186]).

Results and Analysis. Fig. 3 shows the performance (throughput in terms of #queries/sec) of the tools normalized to No-I/O. We make three key observations. First, I/O overhead has a large impact on performance for all cases. Compared to SSD-C (SSD-P), No-I/O leads to $9.4\times$ ($1.7\times$) and $32.9\times$ ($3.6\times$) better performance in R-Qry and S-Qry (averaged across both databases), respectively. While both baselines significantly suffer from large I/O overhead, we observe a relatively larger impact on S-Qry due to its lower data reuse compared to R-Qry. This is because the lower the data reuse, the less effectively the initial I/O cost can be amortized. Second, even using the costly state-of-the-art SSD (SSD-P) does not alleviate this overhead, leaving large performance gaps between SSD-P and No-I/O in

⁴We experiment with both techniques of accessing the database devised in the R-Qry baseline [73] and report the best timing. The first technique uses mmap to access the database, while the second technique loads the entire database from the SSD to DRAM as the first step when the analysis starts. In this experiment, the second approach performs slightly better since, when analyzing our read set, the application accesses most parts of the database.

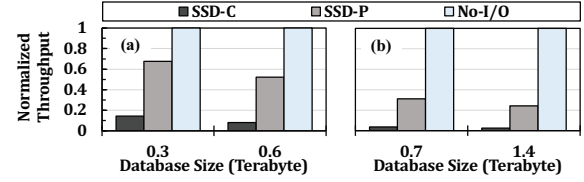


Figure 3: Performance of (a) R-Qry and (b) S-Qry under different storage configurations and database sizes.

both tools. Third, I/O overhead increases as the databases grow. For example, in R-Qry, the performance gap between SSD-C and No-I/O widens from $7.1\times$ to $12.5\times$ as the database expands from 0.3 TB to 0.6 TB. Based on these observations, We conclude that I/O accesses lead to large overheads in metagenomic analysis, an issue expected to worsen in the future.

This I/O overhead, stemming from the need to move large amounts of low-reuse data, is a fundamental problem that is hard to avoid. One might think it is possible to avoid this overhead by 1) using sampling techniques to shrink database sizes (e.g., [48–53]) or 2) keeping all data required by metagenomic analysis completely and always resident in main memory. Neither of these solutions is suitable. The first approach inevitably reduces accuracy [18, 54, 59] to levels unacceptable for many use cases (e.g., [18, 42, 54–59]). The second approach is energy inefficient, costly, unscalable, and unsustainable due to two reasons. First, the sizes of metagenomic databases (which are already large, i.e., in some recent examples, exceeding a hundred terabytes [36, 37]) have been increasing rapidly. For example, recent trends show the *doubling* of different important databases in only several months [25, 40, 42]. Second, regardless of the sizes of individual databases, different analyses need *different databases*, with information from different sets of genomes or with varying parameters. For example, a medical center may use various databases for its patients based on the patients' conditions [172] (e.g., for different viral infections [48, 193], sepsis [11], etc.). Therefore, it is inefficient and unsustainable to maintain *all* data required by *all possible* analyses in DRAM at all times.⁵

The I/O impact on end-to-end performance becomes even more prominent in emerging systems in which other bottlenecks are alleviated. For example, while metagenomics can benefit from near-data processing at the main memory level, i.e., processing-in-memory (PIM) [64, 65, 67, 68, 197–201], these approaches still incur the overhead of moving the large, low reuse data from the storage system. In fact, by alleviating other bottlenecks, the impact of I/O on end-to-end performance increases. For example, for the 0.3-TB and 0.6-TB Kraken2 databases, using a state-of-the-art PIM accelerator [64] of Kraken2, No-I/O is on average $26.1\times$ ($3.0\times$) faster than SSD-C (SSD-P). We conclude that while accelerating other bottlenecks in metagenomic analysis (e.g., main memory bottlenecks) can provide significant benefits, doing so does not alleviate the overheads of moving large, low-reuse data from the storage system.

3.3. Our Goal

ISP can be a fundamental solution for reducing data movement. However, designing an ISP system for metagenomics

⁵Ultimately, these are the same reasons that the metagenomics community has been investigating storage efficiency (e.g., the aforementioned sampling techniques [48–53]) as opposed to merely relying on scaling the system's main memory [35, 57, 194–196].

is challenging because none of the existing approaches can be directly implemented as an ISP system effectively due to an SSD's constrained hardware resources. Techniques such as R-Qry hinder leveraging ISP's large potential by preventing the full utilization of the SSD's internal bandwidth due to costly conflicts in internal SSD resources [79–81] caused by random accesses. Techniques such as S-Qry predominantly incur more suitable streaming accesses, but at the cost of more computation and main memory capacity requirements, posing challenges for ISP. Therefore, directly adopting existing metagenomic analysis approaches in storage incurs performance, energy, and lifetime overheads. **Our goal** in this work is to improve the performance and efficiency of metagenomic analysis by reducing the large data movement overhead from the storage system in a cost-effective manner.

4. MegIS

We propose MegIS, the *first* ISP system designed for the end-to-end metagenomic analysis pipeline to reduce its data movement overheads from the storage system. MegIS is primarily designed as a system for accelerating metagenomic analysis. MegIS extends the existing SSD controller and FTL *without* impacting the baseline SSD functionality. Therefore, when metagenomic acceleration is not in progress, the SSD can be accessible for all other applications, similar to a general-purpose SSD.

We address the challenges of ISP for metagenomic analysis via hardware/software co-design to enable what we call *cooperative ISP*. In other words, we do not solely focus on processing inside the storage system but, instead, we exploit the strengths of processing both inside and outside the storage system. MegIS enables an efficient pipeline between the host system and the storage system to maximally leverage and orchestrate the capabilities of both systems.

It is possible for MegIS's ISP steps to run on our lightweight specialized ISP accelerators or, alternatively, on the existing embedded cores in the SSD controller⁶ or other general-purpose ISP systems (e.g., [105, 143, 157, 159]). This is because, leveraging our optimizations, MegIS's ISP steps require only simple computation and small buffers. Efficiently performing metagenomics on any of these underlying hardware units requires MegIS's specialized task partitioning, data/computation flow coordination, storage technology-aware algorithmic optimizations, and data mapping. The ability to leverage existing hardware units (embedded SSD cores or general-purpose ISP systems) helps with MegIS's ease of adoption. Ultimately, choosing between different MegIS configurations (our specialized lightweight accelerators or general-purpose hardware) is a design decision that has various tradeoffs, with specialized accelerators achieving the highest performance and power efficiency (§6.1).

4.1. Overview

Fig. 4 shows an overview of MegIS's steps. We design MegIS as an efficient pipeline in the SSD and the host system. We develop MegIS FTL (§4.5), which is responsible for communication with the host system and data flow across the SSD hardware components (e.g., NAND flash chips, internal DRAM, and

⁶These cores are available for MegIS's ISP since we envision that during metagenomic acceleration, MegIS is not used as a general-purpose SSD and does not run the baseline FTL. Instead, it runs MegIS FTL, which only performs lightweight and infrequent tasks during ISP (see §4.5).

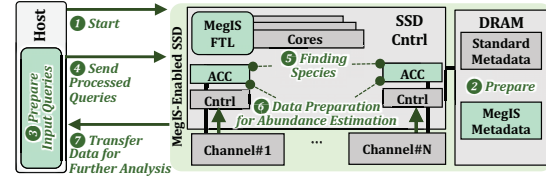


Figure 4: Overview of MegIS.

hardware accelerators) when running metagenomic analysis. Upon receiving a notification from the host⁷ to initiate metagenomic analysis (① in Fig. 4), MegIS readies itself by loading the necessary MegIS FTL metadata (②). After this preparation, MegIS starts its three-step execution. In **Step 1** (§4.2), the host processes the input read queries (③) and transfers them in batches to the SSD (④). In **Step 2** (§4.3), the ISP units (ACC in Fig. 4) find the species present in the sample (⑤). Steps 1 and 2 run in a pipelined manner. In **Step 3** (§4.4), MegIS prepares (⑥) and transfers (⑦) the data needed for any further analysis. By doing so, MegIS facilitates integration with different abundance estimation approaches. MegIS leverages the SSD's *full internal bandwidth* since it avoids channel conflicts (due to its specialized data/control flow) and frequent management tasks (by not requiring writes during its ISP steps).

4.2. Step 1: Preparing the Input Queries

In this step, MegIS prepares the input read queries in a metagenomic sample for metagenomic analysis. MegIS works with lexicographically-sorted data structures to avoid expensive random accesses to the SSD (similar to S-Qry, described in §2.1). Like many other metagenomic tools (e.g., [48, 49, 51–53, 67, 73–78, 82, 83, 90]), we assume the sorted k-mer *databases* are pre-built before the analysis. However, sorting k-mers extracted from the *input query read set* is inefficient to perform offline due to the need to store a large data structure (sorted k-mer set) with each sample, potentially larger than the sample itself, causing significant storage capacity waste. Therefore, to prepare the input queries, MegIS 1) extracts k-mers from the sample (§4.2.1), 2) sorts the k-mers (§4.2.2), and if needed, 3) prunes some k-mers according to user-defined criteria (§4.2.3).

We execute this step in the host system for three reasons. First, this step benefits from the relatively larger DRAM and more powerful computation resources in the host. Second, due to the large host-side DRAM, performing this step in the host leads to significantly fewer writes to the flash chips, positively impacting lifetime. For typical metagenomic read sets, storing k-mers extracted from reads within a sample takes tens of gigabytes (e.g., on average 60 GB with standard CAMI read sets [59]). While generating and sorting k-mers inside the SSD is possible, it would necessitate frequent writes to flash chips or much larger DRAM. Third, by leveraging the host system for this step, we enable pipelining and overlapping Step 1 with Step 2 (which searches the large, low-reuse database).

To efficiently execute Step 1 on the host system, we need to ensure two points. First, partitioning the application between the host system and the SSD should not incur significant overheads due to data transfer time. Second, while it is reasonable in most cases to expect the host DRAM to be large enough to contain all extracted k-mers from a sample, MegIS should accommodate scenarios where this is not the case and mini-

⁷See an extended version [202] for more detail on MegIS's storage interface commands.

mize the performance, lifetime, and endurance overheads of writes to flash chips due to page swaps (i.e., moving data back-and-forth between the host DRAM and the SSD when the host DRAM is smaller than the application's working set size).

The sequences in MegIS's databases are encoded with two bits per character (i.e., A, C, G, T in DNA alphabet) during their offline generation. For the read sets, MegIS is able to work with different formats. We perform the first analysis step (Step 1) in the host system so that any format conversion can be flexibly incorporated there (e.g., from ASCII or binary to 2-bit encoding). The overhead of format conversion is negligible since it involves a straightforward transformation of the four nucleotide bases to the 2-bit encoded format. For the remainder of MegIS's pipeline, we use the 2-bit encoding.

4.2.1. K-mer Extraction. To reduce data transfer overhead between different parts of the application that execute in the host system and in the storage system, we propose a new input processing scheme by improving upon the input processing scheme in KMC [203]. We partition the k-mers into buckets, each corresponding to a lexicographical range. This enables overlapping the k-mer sorting and transfer of a bucket to the SSD with the ISP operations of Step 2 (§4.3) on previously transferred buckets. This is because the database k-mers are also sorted and can already be accessed within the corresponding range. Fig. 5 shows an overview of MegIS's k-mer extraction. The host reads the input reads from the storage system (① in Fig. 5), extracts their k-mers (②), and stores them in the buckets (③).⁸ In situations where a sample's extracted k-mers do not fit in the host DRAM, MegIS pins some buckets to the host DRAM (e.g., Buckets 1 to $N - 1$ in Fig. 5) and uses the SSD to store the others. This way, k-mers belonging to buckets in the host DRAM do *not* move back and forth between the host DRAM and the SSD (④). To reduce the overhead of accessing buckets in the SSD, MegIS takes two measures. First, MegIS allocates buffers in the host DRAM specifically for buckets in the SSD. Once these buffers are full, it efficiently transfers their contents to the SSD, maximizing the use of the sequential-write bandwidth. Second, we map each bucket's k-mers across SSD channels evenly for parallelism. Since MegIS does not require writes to the flash chips after this step (i.e., K-mer Extraction in Step 1), it can flush all of the FTL metadata for write-related management to free up internal DRAM for the next steps (details in §4.5).

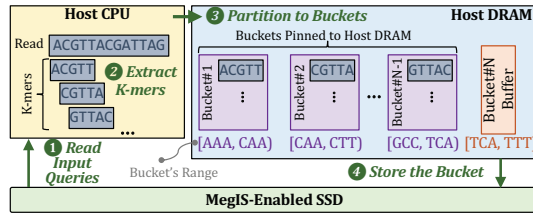


Figure 5: Overview of the k-mer extraction process in MegIS.

4.2.2. Sorting. After generating all k-mer buckets, MegIS proceeds to sort the k-mers within the individual buckets. As soon as a specific bucket i is sorted, MegIS transfers this bucket to the DRAM inside the SSD in batches (to undergo Step 2, as described in §4.3). Meanwhile, during the transfer of bucket i ,

⁸To prevent bucket size imbalance, we initially create preliminary buckets for a small k-mer subset. In case of imbalance, we merge some buckets to satisfy a user-defined bucket count (default 512).

MegIS advances to sort bucket $i + 1$. MegIS can orthogonally use a sorting accelerator (e.g., [204–206]) to perform sorting.

4.2.3. Excluding K-mers. MegIS, like various tools [5, 75, 82, 203, 207, 208], can exclude k-mers based on user-defined frequencies to improve accuracy. Users can exclude 1) overly common (i.e., indiscriminative) k-mers and 2) very infrequent k-mers (e.g., those that appear only once), which may represent sequencing errors or low-abundance organisms that are hard to distinguish from random occurrences. Exclusion follows sorting, where k-mers are already counted. While the size of the extracted query k-mers (§4.2.1) can be large (on average 60 GB in our experiments), the size of the k-mer set selected to go to Step 2 is much smaller (on average 6.5 GB) and is significantly smaller than the database that may reach several terabytes [34–39].

4.3. Step 2: Finding Candidate Species

In Step 2, MegIS finds the species present in the sample by 1) intersecting the query k-mers and the database k-mers, and 2) finding the taxIDs of the intersecting k-mers. We perform this stage inside the SSD since it requires streaming the large database with low reuse and involves only lightweight computation. This enables MegIS to leverage the SSD's large internal bandwidth and alleviate the overall burden of moving/analyzing large, low-reuse data from the rest of the system.

Considering the SSD's hardware limitations, MegIS should leverage the full internal bandwidth without requiring expensive hardware resources inside the SSD (e.g., large internal DRAM size/bandwidth and costly logic units). Performing this step effectively inside the SSD requires efficient coordination between the SSD and the host, mapping, hardware design, and storage technology-aware algorithmic optimizations.

4.3.1. Intersection Finding. In this step, MegIS finds the intersecting k-mers, i.e., k-mers present in both the query k-mer buckets arriving from the host system and the large k-mer database stored in the flash chips.

Relying solely on the SSD's internal DRAM to 1) buffer the query k-mers arriving from the host system and the database k-mers arriving from the SSD channels at full bandwidth and 2) stream through both to find their intersection can pressure the valuable internal DRAM bandwidth. For example, reading the database from the SSD channels at full bandwidth in a high-end SSD can already exceed the LPDDR4 DRAM bandwidth used in current SSDs [84, 105, 209] and even the 16-GB/s DDR4 bandwidth [105, 187]. To address this challenge, we adopt an approach similar to [105] and operate on data fetched from flash chips without buffering them in the internal DRAM. Despite its benefits, this approach requires large buffers (64 KB for input and 64 KB for output) *per channel*.

To facilitate low-cost computation on flash data streams, we leverage two key features of MegIS to find the minimum required buffer size. First, the computation in this step is lightweight and does not require a large buffer for data awaiting computation. Second, data is uniformly spread across channels, with each compute unit handling data from one channel. Based on these, we directly read data from the flash chips and include *two* k-mer registers per channel. One register holds a k-mer as the computation input, while the other register stores the subsequent k-mer as it is read from the flash chips. This way, by only using two registers, MegIS directly computes on the flash data stream at low cost.

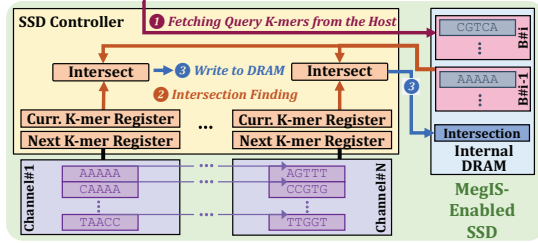


Figure 6: Overview of the intersection finding process in MegIS.

Fig. 6 shows the overview of MegIS’s intersection finding process. First, MegIS reads the query k-mers to the internal DRAM in batches (① in Fig. 6). Second, it concurrently reads both the sorted query k-mers (from the internal DRAM) and the sorted database k-mers (from the flash chips), performing a comparison to find their intersection (②) using per-channel *Intersect* units located in the SSD controller. Third, MegIS writes the intersecting k-mers to the internal DRAM for further analysis (③).

Fetching Query K-mers. To efficiently use external bandwidth, MegIS moves buckets from the host system to the internal DRAM in batches. We manage two batches in the internal DRAM to overlap transfer and intersection finding. For an SSD with 8 channels, 4 dies/channel, 2 planes/die, and 16-KiB pages, MegIS requires space for two 1-MiB batches (i.e., $B\#i - 1$ and $B\#i$ in Fig. 6) in the internal DRAM.

Intersection Finding. MegIS reads the query k-mers from the internal DRAM and the database k-mers from the flash chips. Intersection Finding runs in a pipelined manner with Fetching Query K-mers. We store the database evenly across different channels to leverage the full internal bandwidth when sequentially reading data using multi-plane operations. MegIS finds the intersecting k-mers as follows: If a database k-mer equals a query k-mer, MegIS records the k-mer as an intersecting k-mer. If a query k-mer is larger (smaller), MegIS reads the next database (query) k-mer. MegIS’s Control Unit, located on the SSD controller, receives the comparison results and issues the control signals accordingly.⁹

Storing the Intersecting K-mers. MegIS stores the intersecting k-mers in the SSD’s internal DRAM.¹⁰ The internal DRAM needs to support 1) fetching the queries, 2) reading them out, 3) storing the intersection, and 4) reading FTL metadata. Since the query k-mer set, the intersection, and the FTL metadata (FTL details in §4.5) are significantly smaller than the database, they can be accessed at a much smaller bandwidth than the bandwidth required for reading the database. For example, for our datasets in §5, when fully leveraging SSD-P’s internal flash bandwidth by reading the database from all flash channels, MegIS requires only 2.4 GB/s of DRAM bandwidth to access all datasets stored in the internal DRAM.

4.3.2. Retrieving TaxIDs. MegIS finds the taxIDs of the species corresponding to the intersecting k-mers by looking up the intersecting k-mers in a pre-built sketch database. Each sketch is a small representative subset of k-mers associated with a given taxID. A sketch database stores the k-mer sketches

⁹Figs. 5, 6, and 8 exclude Control Unit and its connections for readability.

¹⁰The intersecting k-mers do not have a strict size requirement and can use the available internal DRAM’s space opportunistically. Usually, its small size allows it to fully fit in the internal DRAM. However, in a case where it does not, MegIS starts the taxID retrieval (§4.3.2) for the already-found intersecting k-mers; then resumes this step, overwriting the old intersecting k-mers.

and their associated taxIDs for a given set of species. Similar to [82], we use CMash [210] to generate sketches. MegIS can also use other sketch generation methods. MegIS flexibly supports variable-sized k-mers in its sketch database. As shown by prior works [48, 210], while longer k-mers are more unique and offer greater discrimination, they may result in missing matches between the intersecting k-mers and sketches. In such cases, users may also search for smaller k-mers by looking up the prefixes of the intersecting k-mers in the sketch database. This enables finding additional matches and increasing the true positive rate.

Finding taxIDs for variable-sized k-mers is challenging since it requires many pointer-chasing operations on a large data structure that may not fit in the SSD’s internal DRAM. To support variable-sized k-mers, some approaches (e.g., [48, 51, 82, 210, 211]) provide data structures to encode the k-mer information in a space-efficient manner. For example, CMash [210] encodes k-mers of variable sizes in a ternary search tree. Fig. 7 shows sketch databases with variable-sized k-mers ($k = 5, 4$, and 3) alongside their taxIDs in **a** separate tables, as used by some prior approaches [90, 212], and **b** in a ternary search tree. This tree structure is devised to 1) save space and 2) retrieve the taxIDs for all k-mers with $k \leq k_{max}$ that are prefixes of a query k_{max} -mer. For example, as shown in Fig. 7, when traversing the tree to look up the 5-mer AATCC, we can look up the 4-mer AATC during the same traversal. Despite its benefits, this approach requires up to k_{max} pointer-chasing operations for *each lookup*. Performing these operations inside the SSD is challenging since the tree can be larger than the SSD’s internal DRAM, and pointer chasing on flash arrays is expensive due to their significantly larger latency compared to DRAM.

While MegIS can perform taxID retrieval in the host system, we identify a new optimization opportunity leveraging unique features of ISP (i.e., large internal bandwidth and storage capacity), which avoids pointer-chasing at the cost of larger data structures. Fig. 7 **c** shows an overview of our approach, *K-mer Sketch Streaming* (KSS). For k-mers with $k = k_{max}$, MegIS stores the k-mer sketches and their taxIDs similar to **a**. MegIS keeps this table in a lexicographically-sorted order. For each smaller k-mer (with $k < k_{max}$), MegIS only stores the taxIDs that are *not* attributed to their corresponding larger, more unique, k-mer. For these smaller k-mers, MegIS does not store the k-mer itself and instead, uses the prefixes of the k_{max} -mers to retrieve the smaller k-mers. MegIS allows for taxID retrieval by sequentially streaming through the intersecting k-mers (which are already sorted) and the KSS tables. While the KSS data structure is larger than the corresponding ternary search tree **b**, it is much more suitable for ISP due to its streaming access feature. KSS can also be efficient for processing outside

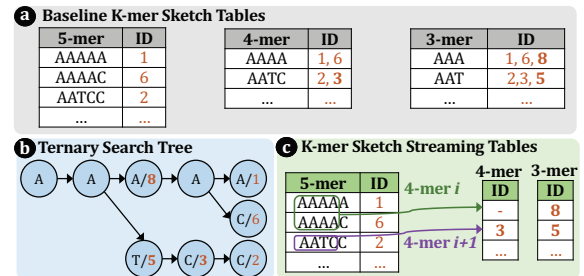


Figure 7: Overview of sketch data structures.

the storage system with SSDs with high external bandwidth (§6.1). KSS leads to $7.5\times$ smaller data structures compared to the 107-GB data structure in **a**, and $2.1\times$ larger compared to **b** (dataset details in §5).

Fig. 8 shows an overview of MegIS's taxID retrieval process. As an example, we demonstrate retrieving 5- and 4-mers. First, MegIS reads the intersecting k-mers (i.e., 5-mers) from the internal DRAM and concurrently reads the 5-mer sketches and their IDs from an SSD channel to find their matches (using the same Intersect unit in §4.3.1) (①). Second, to find 4-mer matches, MegIS compares the *prefixes* of the intersecting 5-mers with the prefixes of the 5-mer sketches (②). MegIS incorporates a lightweight *Index Generator*. It compares the 4-mer prefixes of each pair of consecutive 5-mers. When the prefixes differ (indicating the start of a new 4-mer), it identifies the new prefix as the new 4-mer and reads the next 4-mer taxID from a SSD channel. Third, MegIS sends the retrieved taxIDs to the host (③) as the IDs of the candidate species present in the sample.¹¹

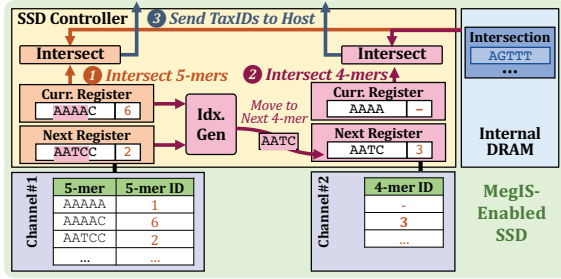


Figure 8: Overview of the TaxID retrieval process in MegIS.

4.4. Step 3: Abundance Estimation

For applications that require abundance estimation, MegIS integrates further analysis on the candidate species identified as present in the sample at the end of Step 2. MegIS can flexibly integrate with different approaches to abundance estimation used in various tools, such as (i) lightweight statistics (e.g., [89–91]) or (ii) more accurate and costly read mapping (e.g., [48, 54, 82]), where the input read set is mapped to the reference genomes of candidate species present in the sample. Based on the relative number of reads that map to each species' reference genome, we can determine the occurrence frequencies of different species. MegIS can integrate with different existing statistical approaches or read mapping, performed in the host or an accelerator, specialized for short reads (e.g., [4, 213, 214]) or long reads (e.g., [4, 7, 213]). We note that Steps 1 and 2 of MegIS are based on k-mers extracted from the reads and do not depend on a specific read length.

While the lightweight statistical approaches can work directly on the output of Step 2, MegIS requires additional data preparation to facilitate read mapping. The read mapper requires the query reads and a unified index of the reference genomes of the candidate species present in the sample [5]. In comparison to using individual indexes for each species, the unified index eliminates the need to search through each index separately, thereby reducing the overheads of the read mapping process. Building indexes for individual species is a one-time task. Yet, creating a unified index for the initially unidentified species present in the sample cannot be done of-

¹¹See an extended version [202] for more detail on MegIS's multi-sample analysis.

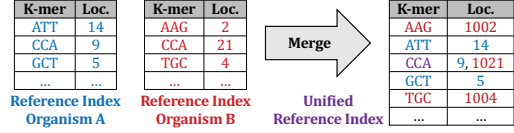


Figure 9: Merging the reference indexes to facilitate read mapping during abundance estimation.

fine. MegIS facilitates index generation for read mapping by generating a unified index in the SSD. Fig. 9 shows an example of the process of unified index generation in MegIS. Each index entry shows a k-mer and its location in that species' reference genome. MegIS reads each index stored in the flash chips sequentially and merges their entries into a unified index. When MegIS finds a common k-mer (e.g., CCA in Fig. 9), it stores the corresponding location of the k-mer in both reference genomes, adjusting the locations with appropriate offsets based on the reference genome sizes. After generating the unified index, MegIS transfers the index to the host system or an accelerator to perform read mapping for abundance estimation.

4.5. MegIS FTL

MegIS FTL needs simple changes to the baseline FTL to handle communication between the host and the SSD.

FTL Metadata. At the beginning of MegIS's operation as a metagenomic acceleration framework, MegIS FTL maintains all metadata of the regular FTL in the internal DRAM. For the only step that requires writes to the NAND flash chips (§4.2.1, K-mer Extraction in the host), MegIS FTL uses the write-related metadata (e.g., L2P, bad-block information). After the K-mer Extraction step, MegIS does not require writes to the NAND flash chips, so it flushes the regular L2P metadata and loads MegIS FTL's L2P metadata while still keeping the other metadata of a regular FTL.

MegIS is designed to only access the underlying flash chips *sequentially*, which inherently reduces the size of the required L2P mapping metadata. In regular FTL, L2P mappings dominate the SSD's internal DRAM capacity due to the page-level granularity of mappings [85, 110, 215]. However, by accessing data sequentially, MegIS FTL circumvents the need for such detailed page-level mappings. Instead, MegIS FTL utilizes a more coarse-grained block-level mapping, which substantially reduces the size of L2P metadata. Therefore, flushing regular L2P mapping metadata into flash chips and using MegIS FTL's metadata enables us to exploit most of the internal DRAM bandwidth and capacity during ISP.

Data Placement. Fig. 10 shows how MegIS FTL manages the target data (i.e., databases¹²) stored in NAND flash with

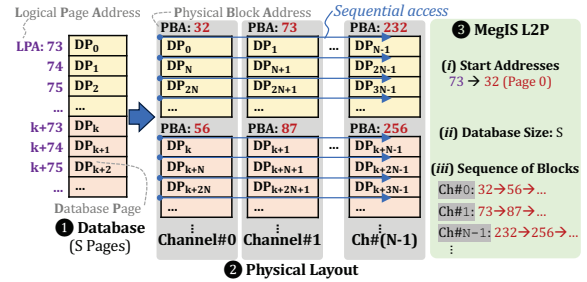


Figure 10: Data layout and mapping data structure in MegIS.

¹²K-mer databases (§4.3.1) and sketch databases (§4.3.1) are the only data structures accessed from NAND flash memory during MegIS's ISP operations.

reduced L2P metadata. When storing a database in the SSD ①, MegIS FTL *evenly and sequentially* distributes the data across all channels ② while ensuring that every *active* block [216] (i.e., blocks available for write operations in the SSD) in different channels has the same page offset. Since MegIS always accesses the database *sequentially*, MegIS FTL’s L2P mapping metadata ③ consists of (i) the mapping between start logical page address (LPA) and physical page address (PPA), (ii) the database size, and (iii) the sequence of physical block addresses storing the database. As shown in Fig. 10, MegIS FTL can sequentially read the stored database from the starting LPA while performing reads in a round-robin manner across channels. To do so, MegIS FTL just increments the PPA within a physical block and resets the PPA when reading the next block.

Compared to the regular L2P, whose space overhead is 0.1% of stored data (4 bytes per 4 KiB), MegIS’s L2P is very small. For example, MegIS only requires ~ 1.3 MB to store a 4-TB database, assuming a physical block size of 12 MB: 4 bytes for each of the 349,525 used blocks (and a few bytes for the start L2P mapping and database size). The only metadata other than L2P that must be kept during ISP is the per-block access count for read-disturbance management [100], so the total MegIS-FTL metadata size is up to 2.6 MB.

SSD Management Tasks. MegIS’s ISP accelerators are located in the SSD controller and access data *after* ECC. ECC does not restrict MegIS’s ISP performance. Modern SSDs are designed with ECC capabilities that match the full internal bandwidth of the SSD to support both I/O requests and internal data migrations due to management tasks like garbage collection [71, 72, 116].

MegIS performs other tasks for ensuring reliability (e.g., refresh to prevent uncorrectable errors [71, 72, 97–104]) *before or after* the ISP because 1) the duration of each MegIS process is significantly smaller than the manufacturer-specified threshold for reliable retention age (e.g., one year [217]), and 2) MegIS avoids read disturbance errors [100] during ISP due to its sequential low-reuse accesses.

5. Evaluation Methodology

Performance. We design a simulator that models all of MegIS’s components, including host operations, accessing flash chips, internal DRAM, in-storage accelerator, and host-SSD interfaces. We feed the latency and throughput of each component to this simulator. For the components in the **hardware-based** steps (e.g., ISP units in Steps 2 and 3): We implement MegIS’s logic components in Verilog. We synthesize them using the Synopsys Design Compiler [218] with a 65 nm library [219] and perform place-and-route using Cadence Innovus [220]. We use two state-of-the-art simulators, Ramulator [221, 222] to model SSD’s internal DRAM, and MQSim [223, 224] to model SSD’s internal operations. For the components in the **software-based** step (e.g., host operations in Step 1), we measure performance on a real system, an AMD® EPYC® 7742 CPU [186] with 128 physical cores and 1-TB DRAM (in all experiments unless stated otherwise). For the software baselines, we measure performance on this real system, with best-performing thread counts. The source code of MegIS, scripts, and datasets can be freely downloaded from <https://github.com/CMU-SAFARI/MegIS>.

SSDs. We use SSD-C [85] and SSD-P [84] as described in §3.2 in our real system experiments. In our MQSim simulations for the

Table 1: SSD configurations.

Specification	SSD-C	SSD-P
General	48-WL-layer 3D TLC NAND flash-based SSD 4 TB capacity, 4 GB internal LPDDR4 DRAM [225]	
Bandwidth (BW)	600 MB/s interface BW (SATA3); 560 MB/s sequential-read BW 1.2-GB/s channel I/O rate	8 GB/s interface BW (4-lane PCIe Gen4); 7 GB/s sequential-read BW 1.2-GB/s channel I/O rate
NAND Config	8 channels, 8 dies/channel, 4 planes/dies, 2,048 blocks/plane, 196 WLS/block, 16 KiB/page (4/8/16 channels in Fig. 16)	16 channels, 8 dies/channel, 2 planes/dies, 2,048 blocks/plane, 196 WLS/block, 16 KiB/page (8/16/32 channels in Fig. 16)
Latencies	Read (tR): 52.5 μ s, Program (tPROG): 700 μ s	
Embedded Cores	3 ARM Cortex-R4 cores [86]	4 ARM Cortex-R4 cores [86]

ISP steps, we faithfully model the SSDs with the configurations summarized in Table 1.

Area and Power. For logic components, we use the results from our Design Compiler synthesis. For SSD power, we use the values of a Samsung 3D NAND flash-based SSD [87]. For DRAM power, we base the values on a DDR4 model [187, 226]. For the CPU cores, we use AMD® μ Prof [227].

Baseline Metagenomic Tools. We use a state-of-the-art performance-optimized (P-Opt) tool, Kraken2 + Bracken [49], and a state-of-the-art accuracy-optimized (A-Opt) tool, Metalign [82]. Particularly, for the presence/absence task, we use Kraken2 without Bracken, and Metalign without mapping (i.e., only KMC [203] + CMash [210]). For abundance estimation, we use Kraken2 + Bracken, and full Metalign. A-Opt achieves significantly higher accuracy compared to P-Opt [59, 82]. In particular, A-Opt leads to $4.8\times$ higher F1 scores and 13% lower L1 norm error on average across all tested inputs. One major reason is that A-Opt uses larger and richer databases compared to performance-optimized P-Opt. MegIS’s end-to-end accuracy matches the accuracy of A-Opt because MegIS’s databases encode the same set of k-mers and sketches as A-Opt.

For both Metalign and MegIS, we use GenCache [213] for mapping. We use the mapping throughput as reported by the original paper [213]. MegIS can be flexibly integrated with other mappers. We also evaluate a state-of-the-art PIM k-mer matching accelerator [64] for accelerating Kraken2’s pipeline. We use the k-mer matching performance as reported by the original paper [64].

Datasets. We use three query read sets from the commonly-used CAMI benchmark [228], with low, medium, and high genetic diversity (i.e., CAMI-L, CAMI-M, and CAMI-H, respectively). Each read set has 100 million reads. We generate a database based on microbial genomes drawn from NCBI’s databases [82, 185] including 155,442 genomes for 52,961 microbial species. For database generation, we use default parameters for each tool. For Kraken2 [49], this results in a 293 GB database. For Metalign [82], this results in a 701 GB k-mer database and a 6.9 GB sketch tree. MegIS uses the same 701 GB k-mer database and a 14 GB sketch database for MegIS’s KSS sketch database (§4.3.2).

6. Evaluation

6.1. Presence/Absence Identification Analysis

We use 1-TB host DRAM in this analysis (smaller than all datasets we evaluate). We examine seven metagenomic anal-

ysis configurations: 1) P-Opt, 2) A-Opt, 3) A-Opt+KSS, where A-Opt leverages the software implementation of MegIS's KSS approach (§4.3.2) instead of Metalign's CMash [82] for retrieving taxIDs, 4) Ext-MS: a MegIS implementation without ISP, where the same accelerators used in MegIS are outside the SSD, 5) MS-NOL: a MegIS implementation *without* overlapping the host and SSD operations as enabled by MegIS's bucketing (§4.2), 6) MS-CC: a MegIS configuration where the SSD cores perform MegIS's ISP tasks, and 7) MS: a MegIS configuration where the hardware accelerators on the SSD controller perform the ISP tasks.

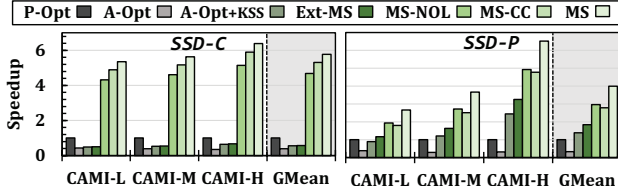


Figure 11: Speedup for different SSDs and input sets.

Fig. 11 shows the speedup of the seven configurations over P-Opt, on three read sets and with SSD-C and SSD-P. We make six key observations. First, MegIS's full implementation (MS) achieves significant speedup compared to both performance-optimized (P-Opt) and accuracy-optimized (A-Opt) baselines. With SSD-C (SSD-P), MS is $5.3\text{--}6.4\times$ ($2.7\text{--}6.5\times$) faster compared to P-Opt, and $12.4\text{--}18.2\times$ ($6.9\text{--}20.4\times$) faster compared to A-Opt. Second, A-Opt+KSS, which leverages MegIS's taxID retrieval approach (KSS) instead of A-Opt's baseline taxID retrieval approach, improves A-Opt's performance by $1.4\times$ ($4.2\times$) on average on SSD-C (SSD-P). MegIS's full implementation outperforms A-Opt+KSS by $10.5\times$ ($2.9\times$). This shows that while MegIS's KSS approach, even outside the SSD, provides large benefits, MegIS's full implementation provides significant additional benefits by alleviating I/O overhead. Third, with SSD-C (SSD-P), MS leads to 23.5% (34.9%) greater average speedup compared to MegIS's implementation without overlapping the steps (MS-NOL). This is due to MegIS's bucketing scheme that enables overlapping the steps. Fourth, MS leads to $10.2\times$ ($2.2\times$) average speedup on SSD-C (SSD-P) compared to MegIS's implementation outside the SSD (Ext-MS) due to the benefits of MegIS's specialized ISP. Fifth, while MS-CC provides large speedup, MS leads to 9% (43%) greater average speedup compared to MS-CC on SSD-C (SSD-P). While both MegIS configurations provide large speedup, this shows that the hardware accelerators are useful and their benefits improve as the internal bandwidth grows. Sixth, MegIS's speedup improves as the genetic diversity of the input read sets increases (from CAMI-L to CAMI-H). This is due to the presence of more species in more diverse read sets, which results in a greater number of sketch tree lookups in the baseline taxID retrieval approach. In contrast, MegIS's KSS efficiently retrieves all taxIDs in a single pass through the sketch tables.

To further demonstrate the benefits of MegIS's optimizations, Fig. 12 shows the time breakdowns with CAMI-L as a representative input. First, KSS improves performance by reducing the execution time of taxID retrieval (as seen by A-Opt+KSS over A-Opt). Second, MegIS without overlapping improves performance over A-Opt+KSS by leveraging ISP to accelerate intersection finding and taxID retrieval (as seen by MS-NOL over A-Opt+KSS). Third, adding overlapping in MegIS's

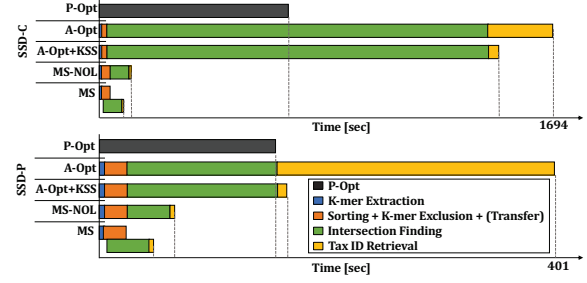


Figure 12: Time breakdown with different SSDs for CAMI-L.

full implementation improves performance by overlapping the execution of sorting in the host system with intersection finding in the SSD (as seen by MS over MS-NOL).

Effect of Database Size. Fig. 13 shows the effect of database size, using CAMI-M as a representative input. The largest database size in each tool (marked by $3\times$) equals the size mentioned in §5. We observe that MegIS's speedups increase as the database size increases (up to $5.6\times/3.7\times$ speedup compared to P-Opt on SSD-C/SSD-P as database size grows to $3\times$).

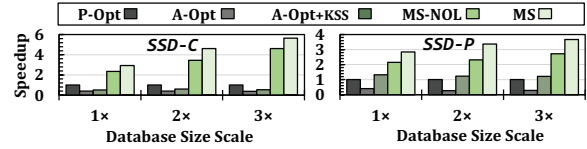


Figure 13: Speedup with different database sizes.

Effect of the Number of SSDs. MegIS benefits from more SSDs in two ways. First, mapping different databases to different SSDs allows for concurrent analyses, each benefiting from MegIS, as already shown (see Fig. 11). Since MegIS's databases and queries are sorted, the database can be disjointly split across SSDs. Fig. 14 demonstrates this case by showing the speedup of different configurations over P-Opt. We show that MegIS maintains its large speedup with many SSDs (i.e., up to eight). As the external bandwidth increases for the baselines (with the number of SSDs), internal bandwidth also increases for MegIS. Particularly, speedup over P-Opt increases until some point (two SSDs) because MegIS takes better advantage of the scaling due to its more efficient streaming accesses. Although there is a slight decrease in speedup when moving from two to eight SSDs, the speedup is still high ($6.9\times/5.2\times$ over eight SSD-Cs/SSD-Ps). This decrease is because in MegIS, due to the large internal bandwidth with 8 SSDs, the overall throughput becomes dependent on the host's sorting. Therefore, in systems with many SSDs, MegIS can be integrated with an accelerator for sorting (e.g., [204–206]) for further speedup. We conclude that MegIS effectively leverages the increased internal bandwidth with more SSDs. Due to this efficient use of multiple SSDs (owing to MegIS's sorted database that can be disjointly partitioned), MegIS can efficiently scale up to very large databases that are distributed across different SSDs.

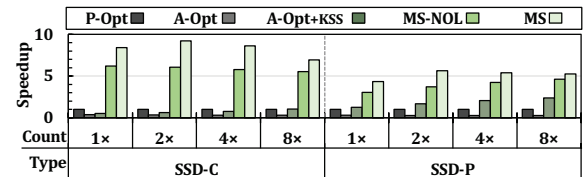


Figure 14: Speedup with different number of SSDs.

Effect of Main Memory Capacity. Fig. 15 demonstrates the effect of host DRAM capacity by showing the speedup of all configurations over P-Opt with CAMI-M.¹³ To gain a fair understanding of I/O overheads when DRAM is smaller than the database, we reduce I/O overheads as much as possible in software. We adopt an optimization [57] to load and process P-Opt’s database into chunks that fit in DRAM.¹⁴ In this case, random accesses to the database in each chunk do not repeatedly access the SSD. However, two overheads still remain: 1) there is still the I/O cost of bringing all chunks from the SSD to the host DRAM, and 2) for every database chunk, all of the input sequences must be queried. We make three observations. First, MegIS’s speedup increases compared to P-Opt with smaller DRAM (e.g., up to $38.5\times$ speedup with 32GB of host DRAM). This is because P-Opt’s performance is hindered by the host DRAM capacity, while MegIS does not rely on large host DRAM. Second, A-Opt and A-Opt+KSS are not affected by the small DRAM (except for the 32-GB configuration) due to their streaming database accesses. But regardless of DRAM size, they suffer from I/O overhead. Third, with the 32-GB DRAM, which is smaller than the extracted query k-mers in Step 1 (§4.2.1), MS’s speedup increases. This is because MegIS’s bucketing (§4.2.1) avoids unnecessary page swaps between the host DRAM and the SSD in this case. We conclude that MegIS enables fast and accurate analysis, *without relying* on large DRAM or large SSD-external bandwidth.

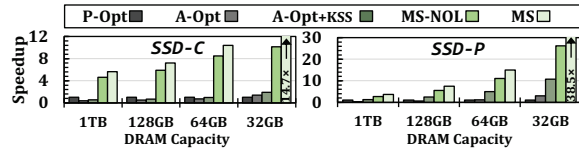


Figure 15: Speedup with different main memory capacities.

Effect of Internal Bandwidth. Fig. 16 shows the effect of internal bandwidth (i.e., by varying the number of SSD channels) on MegIS with CAMI-M as a representative input. We observe that MegIS’s speedup increases as the internal bandwidth increases. On SSD-C (SSD-P), MegIS leads to $12.3\text{--}41.8\times$ ($8.6\text{--}21.6\times$) speedup over A-Opt. The increased speedup is due to the improved performance of MegIS’s ISP steps as the internal bandwidth increases.

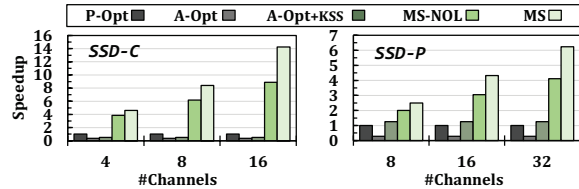


Figure 16: Speedup with varying SSD internal bandwidth.

Impact on System Cost Efficiency. MegIS increases system cost-efficiency because (i) it analyzes large amounts of data inside storage and removes a large part of the analysis burden from other parts of the system, and (ii) it does not rely on either high-bandwidth host-SSD interfaces or large DRAM. Fig. 17 compares MegIS on a cost-optimized system with SSD-C and 64-GB host DRAM (MS_C) to baselines on a performance-optimized system with SSD-P and 1-TB host DRAM (P-Opt_P and A-Opt_P). MS_C provides $2.4\times$ and $7.2\times$ average speedup

¹³In all cases, except for the 32GB configuration, all k-mer buckets extracted from the read set (§4.2.1) fit in the host DRAM.

¹⁴Note A-Opt does not require this due to its streaming database accesses.

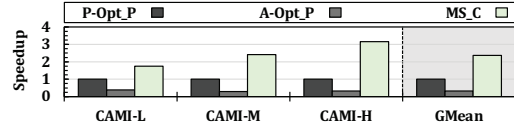


Figure 17: Speedup of MegIS on a cost-optimized system over baselines on a performance-optimized system.

compared to P-Opt and A-Opt, respectively. Note that MS_C provides the same accuracy as A-Opt_P and significantly higher accuracy than P-Opt_P. We conclude that MegIS outperforms the baseline tools even when running on a much less costly system. This is critical to both increasing the system cost-efficiency and enabling portable analysis, which is increasingly important due to the advances of compact portable sequencers [183, 229, 230] for on-site metagenomics [19, 26, 178, 184].

Comparison to a PIM Accelerator. Fig. 18 compares MegIS to a PIM-accelerated baseline. We evaluate Kraken2’s *end-to-end* performance (i.e., including the I/O accesses to load data to the PIM accelerator, k-mer matching, sample classification, and other computation [49]), performing k-mer matching on a state-of-the-art PIM system, Sieve [64].¹⁵ MegIS achieves $4.8\text{--}5.1\times$ ($1.5\text{--}2.7\times$) speedup on SSD-C (SSD-P) over the PIM-accelerated baseline while providing significantly higher accuracy ($4.8\times$ higher F1 scores and 13% lower L1 norm error).¹⁶

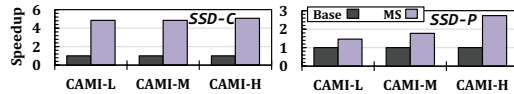


Figure 18: Speedup over a PIM-accelerated baseline [64].

6.2. Abundance Estimation Analysis

We evaluate abundance estimation with four configurations: 1) P-Opt, 2) A-Opt, 3) MS-Nldx: a MegIS implementation that does not leverage MegIS’s third step for generating a unified reference index (§4.2), and instead uses Minimap2 [5] for index generation, and 4) MS: MegIS’s full implementation. Fig. 19 shows speedups over P-Opt. We make two key observations. First, MegIS’s full implementation leads to significant speedup compared to both performance- and accuracy-optimized baselines. MS provides $5.1\text{--}5.5\times$ ($2.5\text{--}3.7\times$) speedup on SSD-C (SSD-P) compared to P-Opt, and $12.0\text{--}15.3\times$ ($6.5\text{--}20.8\times$) speedup compared to A-Opt. Second, MegIS’s full implementation achieves 65% higher average speedup compared to MS-Nldx due to MegIS’s efficient index generation.

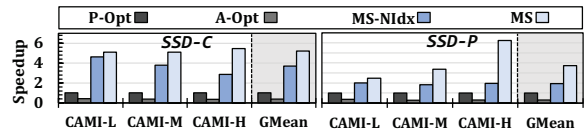


Figure 19: Speedup for abundance estimation.

6.3. Area and Power

Table 2 shows the area and power consumption of MegIS’s hardware accelerator units at 300 MHz. While these units could be designed to operate at a higher frequency, their throughput is already sufficient since MegIS is bottlenecked by NAND flash read throughput. MegIS’s hardware accelerator area and

¹⁵We do not use PIM for Metalign’s k-mer matching as k-mer matching in Metalign is bottlenecked only by I/O bandwidth, not main memory, due to its streaming accesses.

¹⁶A larger database for Kraken2 to encode richer information can increase accuracy for the PIM-accelerated baseline, but with even larger I/O overhead.

Table 2: Area and power consumption of MegIS’s logic.

Logic unit	# of instances	Area [mm ²]	Power [mW]
Intersect (120-bit)	1 per channel	0.001361	0.284
k-mer Registers (2× 120-bit)	1 per channel	0.002821	0.645
Index Generator (64-bit)	1 per channel	0.000272	0.025
Control Unit	1 per SSD	0.000188	0.026
Total for an 8-channel SSD	-	0.04	7.658

power requirements are small: only 0.04 mm² and 7.658 mW at 65 nm. The accelerator can be place-and-routed in a small space with 0.25mm × 0.25mm dimensions (0.0625 mm²). The area overhead of the accelerator is 0.011 mm² at 32 nm,¹⁷ which is 1.7% of the three 28-nm ARM Cortex R4 cores [86] in a SATA SSD controller [87]. While both the accelerator and the cores in the SSD controller can execute MegIS’s ISP tasks, the accelerator is 26.85× more power-efficient than the cores.

6.4. Energy

We demonstrate the energy consumption of different metagenomic analysis tools by obtaining the energy of the host processor, the host DRAM, the accelerators, the SSD’s internal DRAM, host/SSD communications, and the SSD accesses. For each tool, we calculate the energy consumption of each part of the system based on its active/idle power and execution time. We observe that MegIS provides significant energy benefits over other software and hardware baselines by alleviating I/O overhead and reducing the burden of metagenomic analysis in the system (the host processor and DRAM). Across our evaluated SSDs and datasets, MegIS leads to 5.4× (9.8×), 15.2× (25.7×), and 1.9× (3.5×) average (maximum) energy reduction compared to P-Opt, A-Opt, and the PIM-accelerated P-Opt when finding species present in the sample.¹⁸

7. Related Work

To our knowledge, MegIS is the *first* in-storage processing (ISP) system designed to significantly reduce the data movement overhead of the end-to-end metagenomic analysis pipeline. By addressing the challenges of leveraging ISP for metagenomics, MegIS fundamentally alleviates its data movement overhead from the storage system via its efficient and cooperative pipeline between the host and the SSD.

Software Optimization of Metagenomics. Several tools (e.g., [51, 57, 73, 76–78, 82, 90]) use comprehensive databases for high accuracy, but usually incur significant computational and I/O costs. Some tools (e.g., [48–53]) apply sampling to reduce database size, but at the cost of accuracy loss.

Hardware Acceleration of Metagenomics. Several works use GPUs (e.g., [62, 63, 66, 70, 232–234]), FPGAs (e.g., [235–237]), and PIM (e.g., [61, 64, 65, 67, 68, 197]) to accelerate metagenomics by alleviating its computation or main memory overheads. These works do not reduce I/O overheads, whose impact on end-to-end performance becomes even larger when other bottlenecks are alleviated. Some works [238, 239] accelerate metagenomic analysis that use raw genomic signals in targeted sequencing [177, 179, 180, 240]. Targeted sequencing is not a focus of our work since this application looks for specific *known* targets in a sample, while we focus on cases where the contents of the sample are *not known* in advance and require looking up significantly larger databases.

¹⁷We scale area to lower technology nodes using the methodology in [231].

¹⁸See an extended version [202] for additional analyses, including more detailed energy results.

Genome Sequence Analysis. Many works optimize different parts of the genome analysis pipeline [26, 178]. Several works (e.g., [4, 6–8, 214, 241–272]) accelerate read mapping, a commonly-used operation in genomics. As shown in §6.2, MegIS can be seamlessly integrated with different mappers. Some works (e.g., [235, 263, 273, 274]) optimize key primitives such as seeding. While these techniques have the potential to provide several benefits, their adoption in metagenomics requires efficiently dealing with significantly larger and more complex indexes than the ones used in traditional genomics. Therefore, we hope that optimizations introduced in our work can facilitate the future adoption of these seeding techniques in large-scale metagenomics.

In-Storage Processing. Several works propose ISP as accelerators for different applications [106, 107, 111, 112, 114, 122–142] (e.g., in machine learning [106, 111, 112, 140, 275], pattern processing and read mapping [110, 142], k-mer counting [276], and graph analytics [134]). Several works propose ISP in the form of general-purpose processing, inside the storage device [105, 138, 139, 143–154], bulk-bitwise operations using flash [155, 156], SSDs closely integrated with FPGAs [109, 157–161], or with GPUs [162]. None of these works target the end-to-end metagenomic analysis. MegIS has two key differences from prior ISP approaches for genomics (e.g., pattern processing [142], read mapping [110], k-mer counting [276]). First, MegIS is a cooperative ISP system for end-to-end metagenomic analysis that orchestrates processing both inside and outside the storage system, while other approaches focus on a specific task inside the storage system (e.g., pattern matching, read mapping filters, k-mer counting). Second, while a part of MegIS’s pipeline (Part 1 of Step 2) performs the same functionality (i.e., sequence matching) as prior works, MegIS introduces new optimizations due to its unique requirements for cooperative ISP between the SSD and the host. As shown in §4.3.1, these requirements stress the SSD’s limited internal DRAM bandwidth as MegIS must 1) handle data from *both* the host and SSD channels, and 2) share intermediate data across ISP stages efficiently.

8. Conclusion

We introduce MegIS, the *first* in-storage processing system designed to significantly reduce the data movement overhead of end-to-end metagenomic analysis. To enable efficient in-storage processing for metagenomics, we propose new 1) task partitioning, 2) data/computation flow coordination, 3) storage-aware algorithms, 4) data mapping, and 5) lightweight in-storage accelerators. We demonstrate that MegIS greatly improves performance, energy consumption, and system cost efficiency at low area and power costs.

Acknowledgments

We thank the anonymous reviewers of MICRO 2023, HPCA 2024, and ISCA 2024 for feedback. We thank the SAFARI group members for feedback and the stimulating intellectual environment. We acknowledge the generous gifts and support provided by our industrial partners, including Google, Huawei, Intel, Microsoft, and VMware. This research was partially supported by the EU Horizon project BioPIM, the Swiss National Science Foundation (SNSF), Semiconductor Research Corporation (SRC), the ETH Future Computing Laboratory (EFCL), and the AI Chip Center for Emerging Smart Systems Limited (ACCESS).

References

- [1] S. Dusko Ehrlich. MetaHIT: The European Union Project on Metagenomics of the Human Intestinal Tract. *Metagenomics of the Human Body*, 2011.
- [2] Shinichi Sunagawa, Luis Pedro Coelho, Samuel Chaffron, Jens Roat Kultima, Karine Labadie, Guillem Salazar, Bardya Djahanschiri, Georg Zeller, Daniel R. Mende, Adriana Alberti, Francisco M. Cornejo-Castillo, Paul I. Costea, Corinne Craud, Francesco d'Ovidio, Stefan Engelen, Isabel Ferrera, Josep M. Gasol, Lionel Guidi, Falk Hildebrand, Florian Kokoszka, Cyrille Lepoivre, Gipsi Lima-Mendez, Julie Poulain, Bonnie T. Poulos, Marta Royo-Llonch, Hugo Sarmento, Sara Vieira-Silva, Céline Dimier, Marc Picheral, Sarah Seanson, Stefanie Kandels-Lewis, Tara Oceans coordinators, Chris Bowler, Colombar de Vargas, Gabriel Gorsky, Nigel Grimsey, Pascal Hingamp, Daniele Iudicone, Olivier Jaillon, Fabrice Not, Hiroyuki Ogata, Stéphane Pesant, Sabrina Speich, Lars Stemmann, Matthew B. Sullivan, Jean Weissenbach, Patrick Wincker, Eric Karsenti, Jeroen Raes, Silvia G. Acinas, Peer Bork, Emmanuel Boss, Chris Bowler, Michael Follows, Lee Karp-Boss, Uros Krzic, Emmanuel G. Reynaud, Christian Sardet, Mike Sieracki, and Didier Velayoudon. Structure and Function of the Global Ocean Microbiome. *Science*, 2015.
- [3] Noah Fierer. Embracing the unknown: disentangling the complexities of the soil microbiome. *Nature Reviews Microbiology*, 2017.
- [4] Damla Senol Cali, Gurpreet S. Kalsi, Zülal Bingöl, Can Firtina, Lavanya Subramanian, Jeremie S. Kim, Rachata Ausavarungnirun, Mohammed Alser, Juan Gomez-Luna, Amirali Boroumand, Anant Norion, Allison Scibisz, Sreenivas Subramoneyan, Can Alkan, Saugata Ghose, and Onur Mutlu. GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis. In *MICRO*, 2020.
- [5] Heng Li. Minimap2: Pairwise Alignment for Nucleotide Sequences. *Bioinformatics*, 2018.
- [6] Tae Jun Ham, David Bruns-Smith, Brendan Sweeney, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Young H Oh, Krste Asanovic, Jae W Lee, and Lisa Wu Wills. Genesis: A Hardware Acceleration Framework for Genomic Data Analysis. In *ISCA*, 2020.
- [7] Yatish Turakhia, Gill Bejerano, and William J Dally. Darwin: A Genomics Co-processor Provides up to 15,000 x Acceleration on Long Read Assembly. In *ASPLoS*, 2018.
- [8] Saransh Gupta, Mohsen Imani, Behnam Khaleghi, Venkatesh Kumar, and Tajana Rosing. RAPID: A ReRAM Processing In-memory Architecture for DNA Sequence Alignment. In *ISLPED*, 2019.
- [9] Thomas M. Kuntz and Jack A. Gilbert. Introducing the microbiome into precision medicine. *Trends in Pharmacological Sciences*, 2017.
- [10] Matthew Dixon, Maria Stefil, Michael McDonald, Truls Erik Bjerkund-Johansen, Kurt Naber, Florian Wagenlehner, and Vladimir Mouraviev. Metagenomics in diagnosis and improved targeted treatment of UTI. *World Journal of Urology*, 2020.
- [11] Arne M. Taxt, Ekaterina Avershina, Stephan A. Frye, Umaer Naseer, and Rafi Ahmad. Rapid identification of pathogens, antibiotic resistance genes and plasmids in blood cultures by nanopore sequencing. *Scientific Reports*, 2020.
- [12] Ebrahim Afshinnekoo, Cem Meydan, Shanin Chowdhury, Dyala Jaroudi, Collin Boyer, Nick Bernstein, Julia M. Maritz, Darryl Reeves, Jorge Gandara, Sagar Chhangawala, Sofia Ahsanuddin, Amber Simmons, Timothy Nessel, Bharathi Sundares, Elizabeth Pereira, Ellen Jorgensen, Sergios-Orestis Kolokotronis, Nell Kirchberger, Isaac Garcia, David Gandara, Sean Dhanraj, Tanzina Nawrin, Yogesh Saleatore, Noah Alexander, Priyanka Vijay, Elizabeth M. Hénaff, Paul Zumbo, Michael Walsh, Gregory D. O'Mullan, Scott Tighe, Joel T. Dudley, Anya Dunaif, Sean Ennis, Eoghan O'Halloran, Tiago R. Magalhaes, Braden Boone, Angela L. Jones, Theodore R. Muth, Katie Schneider Paolantonio, Elizabeth Alter, Eric E. Schadt, Jeanne Garbarino, Robert J. Prill, Jane M. Carlton, Shawn Levy, and Christopher E. Mason. Geospatial Resolution of Human and Bacterial Diversity with City-scale Metagenomics. *Cell Systems*, 2015.
- [13] Tiffany Hsu, Regina Joice, Jose Vallarino, Galeb Abu-Ali, Erica M. Hartmann, Afrah Shafquat, Casey DuLong, Catherine Baranowski, Dirk Gevers, Jessica L. Green, Xochitl C. Morgan, John D. Spengler, and Curtis Huttenhower. Urban Transit System Microbial Communities Differ by Surface Type and Interaction with Humans and the Environment. *Msystems*, 2016.
- [14] Goldin John, Nikhil Shri Sahajpal, Ashis K. Mondal, Sudha Ananth, Colin Williams, Alka Chaubey, Amyn M. Rojani, and Ravindra Kolhe. Next-Generation Sequencing (NGS) in COVID-19: A Tool for SARS-CoV-2 Diagnosis, Monitoring New Strains and Phylodynamic Modeling in Molecular Epidemiology. *Current Issues in Molecular Biology*, 2021.
- [15] Dorottya Nagy-Szakal, Mara Couto-Rodriguez, Heather L. Wells, Joseph E. Barrows, Marilyne Debieu, Kristin Butcher, Siyuan Chen, Agnes Berki, Courtney Hager, Robert J. Boorstein, Mariah K. Taylor, Colleen B. Jonsson, Christopher E. Mason, and Niamh B. O'Hara. Targeted Hybridization Capture of SARS-CoV-2 and Metagenomics Enables Genetic Variant Discovery and Nasal Microbiome Insights. *Microbiology Spectrum*, 2021.
- [16] David F. Nieuwenhuijse and Marion P. G. Koopmans. Metagenomic sequencing for surveillance of food- and waterborne viral diseases. *Frontiers in Microbiology*, 2017.
- [17] James Hadfield, Colin Megill, Sidney M Bell, John Huddleston, Barney Potter, Charlton Callender, Pavel Sagulenko, Trevor Bedford, and Richard A Neher. Nextstrain: Real-time Tracking of Pathogen Evolution. *Bioinformatics*, 2018.
- [18] Bonnie Berger and Yun William Yu. Navigating bottlenecks and trade-offs in genomic data analysis. *Nature Reviews Genetics*, 2023.
- [19] Augusto Dulanto Chiang and John P Dekker. From the Pipeline to the Bedside: Advances and Challenges in Clinical Metagenomics. *The Journal of Infectious Diseases*, 2019.
- [20] Charles Y. Chiu and Steven A. Miller. Clinical metagenomics. *Nature Reviews Genetics*, 2019.
- [21] Nicholas A. Bokulich, Michal Ziemski, Michael S. Robeson, and Benjamin D. Kaehler. Measuring the microbiome: Best practices for developing and benchmarking microbiomics methods. *Computational and Structural Biotechnology Journal*, 2020.
- [22] Taishan Hu, Nilesh Chitnis, Dimitri Monos, and Anh Dinh. Next-Generation Sequencing Technologies: An Overview. *Human Immunology*, 2021.
- [23] Mohammed Alser, Zülal Bingöl, Damla Senol Cali, Jeremie Kim, Saugata Ghose, Can Alkan, and Onur Mutlu. Accelerating Genome Analysis: A Primer on an Ongoing Journey. *IEEE Micro*, 2020.
- [24] Kenneth Katz, Oleg Shutoy, Richard Lapoint, Michael Kimelman, J Rodney Brister, and Christopher O'Sullivan. The Sequence Read Archive: a decade more of explosive growth. *Nucleic Acids Research*, 2021.
- [25] European Bioinformatics Institute. European Nucleotide Archive Statistics. <https://www.ebi.ac.uk/ena/browser/about/statistics>, 2023.
- [26] Mohammed Alser, Joel Lindegger, Can Firtina, Nour Almadhoun, Haiyu Mao, Gagandeep Singh, Juan Gomez-Luna, and Onur Mutlu. From molecules to genomic variations: Accelerating genome analysis via intelligent algorithms and architectures. *Computational and Structural Biotechnology Journal*, 2022.
- [27] Rasko Leinonen, Hideaki Sugawara, Martin Shumway, and International Nucleotide Sequence Database Collaboration. The Sequence Read Archive. *Nucleic Acids Research*, 2010.
- [28] Gagandeep Singh, Mohammed Alser, Kristof Denolf, Can Firtina, Alireza Khodamrudi, Meryem Banu Cavlak, Henk Corporaal, and Onur Mutlu. Rubicon: a framework for designing efficient deep learning-based genomic basecallers. *Genome Biology*, 2024.
- [29] Zhimeng Xu, Yuting Mai, Denghui Liu, Wenjun He, Xinyuan Lin, Chi Xu, Lei Zhang, Xin Meng, Joseph Mafofo, Walid Abbas Zaher, et al. Fast-bonito: A faster deep learning based basecaller for nanopore sequencing. *Artificial Intelligence in the Life Sciences*, 2021.
- [30] Qian Lou, Sarath Chandra Janga, and Lei Jiang. Helix: Algorithm/architecture co-design for accelerating nanopore genome base-calling. In *PACT*, 2020.
- [31] Taha Shahroodi, Gagandeep Singh, Mahdi Zahedi, Haiyu Mao, Joel Lindegger, Can Firtina, Stephan Wong, Onur Mutlu, and Said Hamdoui. Swordfish: A framework for evaluating deep neural network-based basecalling using computation-in-memory with non-ideal memristors. In *MICRO*, 2023.
- [32] Hiruna Samarakoon, James M Ferguson, Hasindu Gamaarachchi, and Ira W Devoson. Accelerated nanopore basecalling with SLOW5 data format. *Bioinformatics*, 2023.
- [33] Meryem Banu Cavlak, Gagandeep Singh, Mohammed Alser, Can Firtina, Joel Lindegger, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Can Alkan, and Onur Mutlu. Targetcall: Eliminating the Wasted Computation in Basecalling via Pre-Basecalling Filtering. *APBC*, 2023.
- [34] Eric W Sayers, Evan E Bolton, J Rodney Brister, Kathi Canese, Jessica Chan, Donald C Comeau, Catherine M Farrell, Michael Feldgarden, Anna M Fine, Kathryn Funk, Eneida Hatcher, Sivakumar Kannan, Christopher Kelly, Sunghwan Kim, William Klimke, Melissa J Landrum, Stacy Lathrop, Zhiyong Lu, Thomas L Madden, Adriana Malheiro, Aron Marchler-Bauer, Terence D Murphy, Lon Phan, Shashikant Pujar, Sanjida H Rangwala, Valerie A Schneider, Tony Tse, Jiyao Wang, Jian Ye, Barton W Trawick, Kim D Pruitt, and Stephen T Sherry. Database resources of the National Center for Biotechnology Information in 2023. *Nucleic Acids Research*, 2022.
- [35] Mikhail Karasikov, Harun Mustafa, Daniel Danciu, Marc Zimmermann, Christopher Barber, Gunnar Ratsch, and André Kahles. Metagraph: Indexing and Analysing Nucleotide Archives at Petabase-scale. *bioRxiv*, 2020.
- [36] Sergey A Shirayev and Richa Agarwala. Indexing and searching petabyte-scale nucleotide resources. *bioRxiv*, 2023.
- [37] National Center for Biotechnology Information. Introducing pebblescout: Index and search petabyte-scale sequence resources faster than ever. <https://ncbiinsights.ncbi.nlm.nih.gov/2023/09/14/introducing-pebblescout/>, 2023.
- [38] Téó Lemane, Nolan Lezcoche, Julien Lecubin, Eric Pelletier, Magali Lescot, Rayan Chikhi, and Pierre Peterlongo. Indexing and real-time user-friendly queries in terabyte-sized complex genomic datasets with knindex and ora. *Nature Computational Science*, 2024.
- [39] Camille Marchet and Antoine Limasset. Scalable sequence database search using partitioned aggregated Bloom comb trees. *Bioinformatics*, 2023.
- [40] National Center for Biotechnology Information. Re-evaluating the blast nucleotide database (nt). <https://ncbiinsights.ncbi.nlm.nih.gov/2022/11/17/re-evaluating-blast-nucleotide-nt/>, 2022.
- [41] Michael Lynch. Evolution of the mutation rate. *Trends in Genetics*, 2010.
- [42] Daniel J. Nasko, Sergey Koren, Adam M. Phillippy, and Todd J. Treangen. RefSeq database growth influences the accuracy of k-mer-based lowest common ancestor species identification. *Genome Biology*, 2018.
- [43] Nuala A O'Leary, Mathew W Wright, J Rodney Brister, Stacy Ciufio, Diana Haddad, Rich McVeigh, Bhanu Rajput, Barbara Robertse, Brian Smith-White, Danso Ako-Adjei, et al. Reference Sequence (RefSeq) Database at NCBI: Current Status, Taxonomic Expansion, and Functional Annotation. *Nucleic Acids Research*, 2016.
- [44] Jian-Yu Jiao, Lan Liu, Zheng-Shuang Hua, Bao-Zhu Fang, En-Min Zhou, Nimaichand Salam, Brian P Hedlund, and Wen-Jun Li. Microbial dark matter coming to light: challenges and opportunities. *National Science Review*, 2020.
- [45] Wen-Jun Li, Bhagwan Narayan Rekadwad, Jian-Yu Jiao, and Nimaichand Salam. Exploring microbial dark matter and the status of bacterial and archaeal taxonomy: Challenges and opportunities in the future. In *Modern Taxonomy of Bacteria and Archaea: New Methods, Technology and Advances*, 2024.
- [46] Mikko Rautiainen, Sergey Nurk, Brian P. Walenz, Glennis A. Logsdon, David Porubsky, Arang Rhie, Evan E. Eichler, Adam M. Phillippy, and Sergey Koren. Telomere-to-telomere assembly of diploid chromosomes with Verkko. *Nature Biotechnology*, 2023.
- [47] Erich D. Jarvis, Giulio Formenti, Arang Rhie, Andrea Guarracino, Chentao Yang, Jonathan Wood, Alan Tracey, Francoise Thibaud-Nissen, Mitchell R. Vollger, David Porubsky, Haoyu Cheng, Mobin Asri, Glennis A. Logsdon, Paolo Carnevali, Mark J. P. Chaisson, Chen-Shan Chin, Sarah Cody, Joanna Collins, Peter Ebert, Merly Escalona, Olivier Fedrigo, Robert S. Fulton, Lucinda L. Fulton, Shilpa Garg, Jennifer L. Gerton, Jay Ghurye, Anastasiya Granat, Richard E. Green, William Harvey, Patrick Hasenfeld, Alex Hastie, Marina Haukness, Erich B. Jaeger, Miten Jain, Melanie Kirsche, Mikhail Kolmogorov, Jan O. Korbel, Sergey Koren, Jonas Koriach, Joyce Lee, Daofeng Li, Tina Lindsay, Julian Lucas, Feng Luo, Tobias Marschall,

- Matthew W. Mitchell, Jennifer McDaniel, Fan Nie, Hugh E. Olsen, Nathan D. Olson, Trevor Pesout, Tamara Potapova, Daniela Puiu, Allison Regier, Jue Ruan, Steven L. Salzberg, Ashley D. Sanders, Michael C. Schatz, Anthony Schmitt, Valerie A. Schneider, Siddarth Selvaraj, Kishwar Shafin, Alaina Shumate, Nathan O. Stitzel, Catherine Stober, James Torrance, Justin Wagner, Jianxin Wang, Aaron Wenger, Cuanle Xiao, Aleksey V. Zimin, Guojie Zhang, Ting Wang, Heng Li, Erik Garrison, David Haussler, Ira Hall, Justin M. Zook, Evan E. Eichler, Adam M. Phillippy, Benedict Paten, Kerstin Howe, Karen H. Miga, and Human Genome Reference Consortium. Semi-automated assembly of high-quality diploid human reference genomes. *Nature*, 2022.
- [48] Daehwan Kim, Li Song, Florian P. Breitwieser, and Steven L. Salzberg. Centrifuge: Rapid and Sensitive Classification of Metagenomic Sequences. *Genome Research*, 2016.
- [49] Derrick E. Wood, Jennifer Lu, and Ben Langmead. Improved Metagenomic Analysis with Kraken 2. *Genome Biology*, 2019.
- [50] André Müller, Christian Hundt, Andreas Hildebrandt, Thomas Hankeln, and Bertil Schmidt. MetaCache: context-aware classification of metagenomic reads using minhashing. *Bioinformatics*, 2017.
- [51] Li Song and Ben Langmead. Centrifuge: lossless compression of microbial genomes for efficient and accurate metagenomic sequence classification. *Genome Biology*, 2024.
- [52] Alexander T. Dilthey, Chirag Jain, Sergey Koren, and Adam M. Phillippy. Strain-level metagenomic assignment and compositional estimation for long reads with metapaps. *Nature Communications*, 2019.
- [53] Jeremy Fan, Steven Huang, and Samuel D. Chortlton. Bugseq: a highly accurate cloud platform for long-read metagenomic analyses. *BMC Bioinformatics*, 2021.
- [54] Alessio Milanese, Daniel R. Mende, Lucas Paoli, Guillem Salazar, Hans-Joachim Ruscheweyh, Miguelangel Cuenca, Pascal Hingamp, Renato Alves, Paul I. Costea, Luis Pedro Coelho, Thomas S. B. Schmidt, Alexandre Almeida, Alex L. Mitchell, Robert D. Finn, Jaime Huerta-Cepas, Peer Bork, Georg Zeller, and Shinichi Sunagawa. Microbial abundance, activity and population genomic profiling with mOTUs2. *Nature Communications*, 2019.
- [55] Steven L. Salzberg, Florian P. Breitwieser, Anupama Kumar, Haiping Hao, Peter Burger, Fausto J. Rodriguez, Michael Lim, Alfredo Quiñones-Hinojosa, Gary L. Gallia, Jeffrey A. Tornheim, Michael T. Melia, Cynthia L. Sears, and Carlos A. Pardo. Next-generation sequencing in neuropathologic diagnosis of infections of the nervous system. *Neurology - Neuroimmunology Neuroinflammation*, 2016.
- [56] Abraham Gihawi, Yuchen Ge, Jennifer Lu, Daniela Puiu, Amanda Xu, Colin S. Cooper, Daniel S. Brewer, Mihaela Perlea, and Steven L. Salzberg. Major data analysis errors invalidate cancer microbiome findings. *mBio*, 2023.
- [57] Christopher Pockrandt, Aleksey V. Zimin, and Steven L. Salzberg. Metagenomic classification with KrakenUniq on low-memory computers. *Journal of Open Source Software*, 2022.
- [58] Joel Ackelsberg, Jennifer Rakeman, Scott Hughes, Jeannine Petersen, Paul Mead, Martin Schriefer, Luke Kingry, Alex Hoffmaster, and Jay E. Gee. Lack of evidence for plague or anthrax on the new york city subway. *Cell Systems*, 2015.
- [59] Fernando Meyer, Adrian Fritz, Zhi-Luo Deng, David Koslicki, Till Robin Lesker, Alexey Gurevich, Gary Robertson, Mohammed Alser, Dmitry Antipov, Francesco Beghini, Denis Bertrand, Jaqueline J. Brito, C. Titus Brown, Jan Buchmann, Aydin Buluç, Bo Chen, Rayan Chikhi, Philip T. L. C. Clausen, Alexandru Cristian, Piotr Wojciech Dabrowski, Aaron E. Darling, Rob Egan, Eleazar Eskin, Evangelos Georganas, Eugene Goltsman, Melissa A. Gray, Lars Hestbjerg Hansen, Steven Hofmeyer, Pingqin Huang, Luiz Irber, Huijue Jia, Tue Sparholt Jørgensen, Silas D. Kieser, Terje Klemetsen, Axel Kola, Mikhail Kolmogorov, Anton Korobeynikov, Jason Kwan, Nathan LaPierre, Claire Lemaitre, Chenhao Li, Antoine Limasset, Fabio Malcher-Miranda, Serghei Mangul, Vanessa R. Marcelino, Camille Marchet, Pierre Marjion, Dmitry Meleshko, Daniel R. Mende, Alessio Milanese, Niranjan Nagarajan, Jakob Nissen, Sergey Nurk, Leonid Olikier, Lucas Paoli, Pierre Peterlongo, Vitor C. Piro, Jacob S. Porter, Simon Rasmussen, Evan R. Rees, Knut Reinert, Bernhard Renard, Espen Mikal Robertsen, Gail L. Rosen, Hans-Joachim Ruscheweyh, Varuni Sarwal, Nicola Segata, Enrico Seiler, Lizhen Shi, Fengzhu Sun, Shinichi Sunagawa, Søren Johannes Sørensen, Asleigh Thomas, Chengxuan Tong, Mirko Trajkovski, Julien Tremblay, Gherman Uritskiy, Riccardo Vicedomini, Zhengyang Wang, Ziye Wang, Zhong Wang, Andrew Warren, Nils Peder Willassen, Katherine Yelick, Ronghui You, Georg Zeller, Zhengqiao Zhao, Shanfeng Zhu, Jie Zhu, Ruben Garrido-Oter, Petra Gastmeier, Stéphane Hacquard, Susanne Häußler, Ariane Khaledi, Friederike Maechler, Fantin Mesny, Simona Radutoiu, Paul Schulze-Lefert, Nathiana Smit, Till Strowig, Andreas Bremges, Alexander Sczyrba, and Alice Carolyn McHardy. Critical assessment of metagenome interpretation: the second round of challenges. *Nature Methods*, 2022.
- [60] Mohammed Alser, Jeremy Rotman, Dhriti Deshpande, Kodi Taraszka, Huwenbo Shi, Pelin Icer Baykal, Harry Taeyun Yang, Victor Xue, Sergey Knyazev, Benjamin D. Singer, Brunilda Balliu, David Koslicki, Pavel Skums, Alex Zelikovsky, Can Alkan, Onur Mutlu, and Serghei Mangul. Technology Dictates Algorithms: Recent Developments in Read Alignment. *Genome Biology*, 2021.
- [61] Zuher Jahshan, Itay Merlin, Esteban Garzón, and Leonid Yavits. Dash-cam: Dynamic approximate search content addressable memory for genome classification. In *MICRO*, 2023.
- [62] Robin Kobus, André Müller, Daniel Jünger, Christian Hundt, and Bertil Schmidt. MetaCache-GPU: ultra-fast metagenomic classification. In *ICPP*, 2021.
- [63] Xuebin Wang, Taifu Wang, Zhihao Xie, Youjin Zhang, Shiqiang Xia, Ruixue Sun, Xinqiu He, Ruizhi Xiang, Qiwen Zheng, Zhencheng Liu, Jin An Wang, Honglong Wu, Xiangqian Jin, Weijun Chen, Dongfang Li, and Zengquan He. GPMeta: a GPU-accelerated method for ultrarapid pathogen identification from metagenomic sequences. *Briefings in Bioinformatics*, 2023.
- [64] Lingxi Wu, Rasool Sharifi, Marzieh Lenjani, Kevin Skadron, and Ashish Venkat. Sieve: Scalable in-situ DRAM-based accelerator designs for massively parallel k-mer matching. In *ISCA*, 2021.
- [65] Robert Hanhan, Esteban Garzón, Zuher Jahshan, Adam Teman, Marco Lanuzza, and Leonid Yavits. EDAM: edit distance tolerant approximate matching content addressable memory. In *ISCA*, 2022.
- [66] Robin Kobus, Christian Hundt, André Müller, and Bertil Schmidt. Accelerating Metagenomic Read Classification on CUDA-enabled GPUs. *BMC Bioinformatics*, 2017.
- [67] Taha Shahroodi, Mahdi Zahedi, Abhairaj Singh, Stephan Wong, and Said Hamdioui. KrakenOnMem: a memristor-augmented HW/SW framework for taxonomic profiling. In *ICS*, 2022.
- [68] Taha Shahroodi, Mahdi Zahedi, Can Firtina, Mohammed Alser, Stephan Wong, Onur Mutlu, and Said Hamdioui. Demeter: A fast and energy-efficient food profiler using hyperdimensional computing in memory. *IEEE Access*, 2022.
- [69] George Armstrong, Cameron Martino, Justin Morris, Behnam Khaleghi, Jaeyoung Kang, Jeff DeReus, Qiyun Zhu, Daniel Roush, Daniel McDonald, Antonio Gonzalez, et al. Swapping metagenomics preprocessing pipeline components offers speed and sensitivity increases. *Msystems*, 2022.
- [70] Peng Jia, Liming Xuan, Lei Liu, and Chaochun Wei. Metabing: Using gpus to accelerate metagenomic sequence classification. *PLOS ONE*, 2011.
- [71] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Error Characterization, Mitigation, and Recovery in Flash-Memory-Based Solid-state Drives. *IEEE*, 2017.
- [72] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Reliability Issues in Flash-memory-based Solid-state Drives: Experimental Analysis, Mitigation, Recovery. *Inside Solid State Drives*, 2018.
- [73] Derrick E. Wood and Steven L. Salzberg. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, 2014.
- [74] Duy Tin Truong, Eric A. Franzosa, Timothy L. Tickle, Matthias Scholz, George Weingart, Edoardo Pasolli, Adrian Tett, Curtis Huttenhower, and Nicola Segata. MetaPhlAn2 for Enhanced Metagenomic Taxonomic Profiling. *Nature Methods*, 2015.
- [75] Rachid Ounit, Steve Wanamaker, Timothy J. Close, and Stefano Lonardi. CLARK: Fast and Accurate Classification of Metagenomic and Genomic Sequences Using Discriminative K-mers. *BMC Genomics*, 2015.
- [76] Vitor C. Piro, Martin S. Lindner, and Bernhard Y. Renard. DUDes: a top-down taxonomic profiler for metagenomics. *Bioinformatics*, 2016.
- [77] Vitor C. Piro, Temesgen H. Dadi, Enrico Seiler, Knut Reinert, and Bernhard Y. Renard. ganon: precise metagenomics classification against large and up-to-date sets of reference sequences. *Bioinformatics*, 2020.
- [78] Vanessa R. Marcelino, Philip T. L. C. Clausen, Jan P. Buchmann, Michelle Wille, Jonathan R. Iredell, Wieland Meyer, Ole Lund, Tania C. Sorrell, and Edward C. Holmes. Ccmetagen: comprehensive and accurate identification of eukaryotes and prokaryotes in metagenomic data. *Genome Biology*, 2020.
- [79] Rakesh Nadig, Mohammad Sadrosadati, Haiyu Mao, Nika Mansouri Ghiasi, Arash Tavakkol, Jisung Park, Hamid Sarbazi-Azad, Juan Gómez-Luna, and Onur Mutlu. Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses. In *ISCA*, 2023.
- [80] Arash Tavakkol, Mohammad Sadrosadati, Saugata Ghose, Jeremiah Kim, Yixin Luo, Yaohua Wang, Nika Mansouri Ghiasi, Lois Orosa, Juan Gómez-Luna, and Onur Mutlu. FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives. In *ISCA*, 2018.
- [81] Jiho Kim, Seokwon Kang, Yongjun Park, and John Kim. Networked SSD: Flash Memory Interconnection Network for High-Bandwidth SSD. In *MICRO*, 2022.
- [82] Nathan LaPierre, Mohammed Alser, Eleazar Eskin, David Koslicki, and Serghei Mangul. MetaIn: Efficient Alignment-based Metagenomic Profiling Via Containment Min Hash. *Genome Biology*, 2020.
- [83] Wei Shen, Hongyan Xiang, Tianquan Huang, Hui Tang, Mingli Peng, Dachuan Cai, Peng Hu, and Hong Ren. KMCP: accurate metagenomic profiling of both prokaryotic and viral populations by pseudo-mapping. *Bioinformatics*, 2022.
- [84] Samsung. Samsung SSD PM1735. <https://www.samsung.com/semiconductor/ssd/enterprise-ssd/MZPLJ3T2HBJR-00007/>, 2020.
- [85] Samsung. Samsung SSD 870 EVO. <https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/870evo/>, 2021.
- [86] ARM Holdings. Cortex-R4. <https://developer.arm.com/ip-products/processors/cortex-r/cortex-r4>, 2011.
- [87] Samsung. Samsung SSD 860 PRO. <https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/860pro/>, 2018.
- [88] Zheng Sun, Shi Huang, Meng Zhang, Qiyun Zhu, Niina Haiminen, Anna Paola Carrieri, Yoshiki Vázquez-Baeza, Laxmi Parida, Ho-Cheol Kim, Rob Knight, and Yang-Yu Liu. Challenges in benchmarking metagenomic profilers. *Nature Methods*, 2021.
- [89] Jennifer Lu, Florian P. Breitwieser, Peter Thielen, and Steven L. Salzberg. Bracken: Estimating Species Abundance in Metagenomics Data. *PeerJ Computer Science*, 2017.
- [90] David Koslicki and Daniel Falush. MetaPalette: a k-mer Painting Approach for Metagenomic Taxonomic Profiling and Quantification of Novel Strain Variation. *mSystems*, 2016.
- [91] Evangelos A. Dimopoulos, Alberto Carmagnini, Irina M. Velsko, Christina Warinner, Greger Larson, Laurent A. F. Frantz, and Evan K. Irving-Pease. Haystack: A bayesian framework for robust and rapid species identification in high-throughput sequencing data. *PLOS Computational Biology*, 2022.
- [92] Yu Cai, Onur Mutlu, Erich F. Haratsch, and Ken Mai. Program Interference in MLC NAND Flash Memory: Characterization, Modeling, and Mitigation. In *ICCD*, 2013.
- [93] Guiqiang Dong, Ningde Xie, and Tong Zhang. On the Use of Soft-Decision Error-Correction Codes in NAND Flash Memory. *JCAS*, 2010.
- [94] Kai Zhao, Wenzhe Zhao, Hongbin Sun, Xiaodong Zhang, Nanning Zheng, and Tong Zhang. LDPC-in-SSD: Making Advanced Error Correction Codes Work Effectively in Solid State Drives. In *FAST*, 2013.
- [95] Raj Chandra Bose and Dwijendra K. Ray-Chaudhuri. On a Class of Error Correcting Binary Group Codes. *Information and control*, 1960.
- [96] Jiadong Wang, Kasra Vakilinia, Tsung-Yi Chen, Thomas Courtade, Guiqiang Dong, Tong Zhang, Hari Shankar, and Richard Wesel. Enhanced Precision through Multiple Reads for LDPC Decoding in Flash Memories. *JSAC*, 2014.

- [97] Yu Cai, Saugata Ghose, Yixin Luo, Ken Mai, Onur Mutlu, and Erich F. Haratsch. Vulnerabilities in MLC NAND Flash Memory Programming: Experimental Analysis, Exploits, and Mitigation Techniques. In *HPCA*, 2017.
- [98] Yixin Luo, Saugata Ghose, Yu Cai, Erich F Haratsch, and Onur Mutlu. Improving 3D NAND Flash Memory Lifetime by Tolerating Early Retention Loss and Process Variation. *POMACS*, 2018.
- [99] Yixin Luo, Saugata Ghose, Yu Cai, Erich F. Haratsch, and Onur Mutlu. HeatWatch: Improving 3D NAND Flash Memory Device Reliability by Exploiting Self-recovery and Temperature Awareness. In *HPCA*, 2018.
- [100] Yu Cai, Yixin Luo, Saugata Ghose, and Onur Mutlu. Read Disturb Errors in MLC NAND Flash Memory: Characterization, Mitigation, and Recovery. In *DSN*, 2015.
- [101] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F Haratsch, Adrian Crista, Osman S Unsal, and Ken Mai. Error Analysis and Management for MLC NAND Flash Memory. *Intel Technology*, 2013.
- [102] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F Haratsch, Adrian Crista, Osman S Unsal, and Ken Mai. Flash Correct-and-refresh: Retention-aware Error Management for Increased Flash Memory lifetime. In *ICCD*, 2012.
- [103] Keonsoo Ha, Jaeyong Jeong, and Jihong Kim. An Integrated Approach for Managing Read Disturbs in High-density NAND Flash Memory. *TCAD*, 2015.
- [104] Yixin Luo, Yu Cai, Saugata Ghose, Jongmoo Choi, and Onur Mutlu. WARM: Improving NAND Flash Memory Lifetime with Write-Hotness Aware Retention Management. In *MSST*, 2015.
- [105] Chen Zou and Andrew A Chien. ASSASIN: Architecture Support for Stream Computing to Accelerate Computational Storage. In *MICRO*, 2022.
- [106] Siqi Li, Fengbin Tu, Liu Liu, Jilan Lin, Zheng Wang, Yangwook Kang, Yufei Ding, and Yuan Xie. ECSSD: Hardware/Data Layout Co-Designed In-Storage-Computing Architecture for Extreme Classification. In *ISCA*, 2023.
- [107] Vikram Sharma Maitlody, Zaid Qureshi, Weixin Liang, Ziyang Feng, Simon Garcia De Gonzalo, Yujie Li, Hubertus Franke, Junjun Xiong, Jian Huang, and Wen-mei Hwu. Deepstore: In-storage Acceleration for Intelligent Queries. In *MICRO*, 2019.
- [108] Seonyoung Kang, Jiyoung An, Jinpyo Kim, and Sang-Woo Jun. MithrilLog: Near-storage accelerator for high-performance log analytics. In *MICRO*, 2021.
- [109] Gunjae Koo, Kiran Kumar Matam, I Te, HV Krishna Giri Narra, Jing Li, Hung-Wei Tseng, Steven Swanson, and Murali Annaram. Summarizer: Trading Communication with Computing Near Storage. In *MICRO*, 2017.
- [110] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alser, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. GenStore: A High-Performance in-Storage Processing System for Genome Sequence Analysis. In *ASPLOS*, 2022.
- [111] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *HPCA*, 2024.
- [112] Hongsun Jang, Jaeyong Song, Jaewon Jung, Jaeyoung Park, Youngsok Kim, and Jinho Lee. Smart-Infinity: Fast Large Language Model Training using Near-Storage Processing on a Real System. In *HPCA*, 2024.
- [113] Junkyum Kim, Myeonggu Kang, Yunki Han, Yang-Gon Kim, and Lee-Sup Kim. OptimStore: In-Storage Optimization of Large Scale DNNs with On-Die Processing. In *HPCA*, 2023.
- [114] Cangyuan Li, Ying Wang, Cheng Liu, Shengwen Liang, Huawei Li, and Xiaowei Li. GLIST: Towards In-Storage Graph Learning. In *ATC*, 2021.
- [115] AnandTech. New Enterprise SSD Controllers. <https://www.anandtech.com/show/16275/new-enterprise-ssd-controllers-from-silicon-motion-phison-fadu>.
- [116] Jiho Kim, Myeongsoo Jung, and John Kim. Decoupled SSD: Rethinking SSD Architecture through Network-based Flash Controllers. In *ISCA*, 2023.
- [117] Arash Tavakkol, Mohammad Arjomand, and Hamid Sarbazi-Azad. Design for scalability in enterprise SSDs. In *PACT*, 2014.
- [118] Myungsuk Kim, Jisung Park, Genhee Cho, Yoona Kim, Lois Orosa, Onur Mutlu, and Jihong Kim. Evanescence: Architectural Support for Efficient Data Sanitization in Modern Flash-Based Storage Systems. In *ASPLOS*, 2020.
- [119] Jisung Park, Jaeyong Jeong, Sungjin Lee, Youngsun Song, and Jihong Kim. Improving Performance and Lifetime of NAND Storage Systems Using Relaxed Program Sequence. In *DAC*, 2016.
- [120] Jisung Park, Youngdon Jung, Jonghoon Won, Minji Kang, Sungjin Lee, and Jihong Kim. RansomeBlocker: a Low-Overhead Ransomware-Proof SSD. In *DAC*, 2019.
- [121] Li-Pin Chang. On Efficient Wear Leveling for Large-scale Flash-memory Storage Systems. In *SAC*, 2007.
- [122] Shengwen Liang, Ying Wang, Youyou Lu, Zhe Yang, Huawei Li, and Xiaowei Li. Cognitive SSD: A Deep Learning Engine for In-Storage Data Retrieval. In *ATC*, 2019.
- [123] Minsub Kim and Sungjin Lee. Reducing tail latency of DNN-based recommender systems using in-storage processing. In *APSys*, 2020.
- [124] Minje Lim, Jeeyoon Jung, and Dongkun Shin. LSM-Tree Compaction Acceleration Using In-Storage Processing. In *ICCE-Asia*, 2021.
- [125] Jianguo Wang, Dongchul Park, Yang-Suk Kee, Yannis Papakonstantinou, and Steven Swanson. SSD in-storage computing for list intersection. In *DaMoN*, 2016.
- [126] Sung-Tae Lee and Jong-Ho Lee. Neuromorphic computing using NAND flash memory architecture with pulse width modulation scheme. *Frontiers in Neuroscience*, 2020.
- [127] Myeonggu Kang, Hyeonuk Kim, Hyein Shin, Jaehyeong Sim, Kyeonghan Kim, and Lee-Sup Kim. S-FLASH: A NAND flash-based deep neural network accelerator exploiting bit-level sparsity. *TC*, 2021.
- [128] Runze Han, Yachen Xiang, Peng Huang, Yihao Shan, Xiaoyan Liu, and Jinfeng Kang. Flash memory array for efficient implementation of deep neural networks. *Advanced Intelligent Systems*, 2021.
- [129] Shaodi Wang. MemCore: Computing-in-Flash Design for Deep Neural Network Acceleration. In *EDTM*, 2022.
- [130] Panni Wang, Feng Xu, Bo Wang, Bin Gao, Huaqiang Wu, He Qian, and Shimeng Yu. Three-dimensional NAND flash for vector-matrix multiplication. In *VLSI*, 2018.
- [131] Runze Han, Peng Huang, Yachen Xiang, Chen Liu, Zhen Dong, Zhiqiang Su, Yongbo Liu, Lu Liu, Xiaoyan Liu, and Jinfeng Kang. A novel convolution computing paradigm based on NOR flash array with high computing speed and energy efficiency. *TCAS-I*, 2019.
- [132] Won Ho Choi, Pi-Feng Chiu, Wen Ma, Gertjan Hemink, Tung Thanh Hoang, Martin Lueker-Boden, and Zvonimir Bandic. An in-flash binary neural network accelerator with SLC NAND flash array. In *ISCA*, 2020.
- [133] Shuyi Pei, Jing Yang, and Qing Yang. REGISTOR: A Platform for Unstructured Data Processing inside SSD Storage. *TOS*, 2019.
- [134] Sang-Woo Jun, Andy Wright, Sizhuo Zhang, Shuotao Xu, and Arvind. GraBoost: Using Accelerated Flash Storage for External Graph Analytics. In *ISCA*, 2018.
- [135] Jaeyoung Do, Yang-Suk Kee, Jignesh M Patel, Chanik Park, Kwanghyun Park, and David J DeWitt. Query Processing on Smart SSDs: Opportunities and Challenges. In *SIGMOD*, 2013.
- [136] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. Willow: A User-Programmable SSD. In *OSDI*, 2014.
- [137] Sunghwan Kim, Hyunok Oh, Chanik Park, Sangyeun Cho, Sang-Won Lee, and Bongki Moon. In-storage Processing of Database Scans and Joins. *Information Sciences*, 2016.
- [138] Erik Riedel, Christos Faloutsos, Garth A Gibson, and David Nagle. Active Disks for Large-Scale Data Processing. *Computer*, 2001.
- [139] Erik Riedel, Garth Gibson, and Christos Faloutsos. Active Storage for Large-Scale Data Mining and Multimedia Applications. In *Vldb*, 1998.
- [140] Yunjae Lee, Jinha Chung, and Minsoo Rhu. SmartSAGE: training large-scale graph neural networks using in-storage processing architectures. In *ISCA*, 2022.
- [141] Won Seob Jeong, Changmin Lee, Keunsoo Kim, Myung Kuk Yoon, Won Jeon, Myoungsoo Jung, and Won Woo Ro. REACT: Scalable and High-performance Regular Expression Pattern Matching Accelerator for In-storage Processing. *TPDS*, 2019.
- [142] Sang-Woo Jun, Huy T. Nguyen, Vijay Gadepally, and Arvind. In-storage Embedded Accelerator for Sparse Pattern Processing. In *HPEC*, 2016.
- [143] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoo Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaehoon Jeong, and Duckhyun Chang. Biscuit: A Framework for Near-data Processing of Big Data Workloads. In *ISCA*, 2016.
- [144] Yangwook Kang, Yang-suk Kee, Ethan L Miller, and Chanik Park. Enabling Cost-effective Data Processing with Smart SSD. In *MSST*, 2013.
- [145] Xiaohao Wang, Yifan Yuan, You Zhou, Chance C Coats, and Jian Huang. Project Almanac: A Time-traveling Solid-state Drive. *EuroSys*, 2019.
- [146] Anurag Acharya, Mustafa Uysal, and Joel Saltz. Active Disks: Programming Model, Algorithms and Evaluation. In *ASPLOS*, 1998.
- [147] Kimberly Keeton, David A Patterson, and Joseph M Hellerstein. A Case for Intelligent Disks (IDISks). *SIGMOD Record*, 1998.
- [148] Farnood Merrikh-Bayat, Xinjie Guo, Michael Klachko, Mirko Prezioso, Konstantin K Likharev, and Dmitri B Strukov. High-performance mixed-signal neurocomputing with nanoscale floating-gate memory cell arrays. *TNNLS*, 2017.
- [149] Devesh Tiwari, Simona Boboila, Sudharshan Vazhkudai, Youngjae Kim, Xiaosong Ma, Peter Desnoyers, and Yan Solihin. Active flash: Towards energy-efficient, in-situ data analytics on extreme-scale machines. In *FAST*, 2013.
- [150] Devesh Tiwari, Sudharshan S Vazhkudai, Youngjae Kim, Xiaosong Ma, Simona Boboila, and Peter J Desnoyers. Reducing data movement costs using energy-efficient, active computation on ssd. In *HotPower*, 2012.
- [151] Simona Boboila, Youngjae Kim, Sudharshan S Vazhkudai, Peter Desnoyers, and Galen M Shipman. Active flash: Out-of-core data analytics on flash storage. In *MSST*, 2012.
- [152] Duck-Ho Bae, Jin-Hyung Kim, Sang-Wook Kim, Hyunok Oh, and Chanik Park. Intelligent SSD: a turbo for big data mining. In *CIKM*, 2013.
- [153] Mahdi Torabzadehkashi, Siavash Rezaei, Vladimir Alves, and Nader Bagherzadeh. Compstor: An in-storage computation platform for scalable distributed processing. In *IPDPSW*, 2018.
- [154] Luyi Kang, Yuqi Xue, Weiwei Jia, Xiaohao Wang, Jongryool Kim, Changhwan Yoon, Myeong Joon Kang, Hyung Jin Lim, Bruce Jacob, and Jian Huang. Iceclav: A trusted execution environment for in-storage computing. In *MICRO*, 2021.
- [155] Congming Gao, Xin Xin, Youyou Lu, Youtao Zhang, Jun Yang, and Jiwei Shu. ParaBit: processing parallel bitwise operations in NAND flash memory based SSDs. In *MICRO*, 2021.
- [156] Jisung Park, Roknoddin Azizi, Geraldo F Oliveira, Mohammad Sadrosadati, Rakesh Nadig, David Novo, Juan Gómez-Luna, Myungsuk Kim, and Onur Mutlu. FlashCosmos: In-Flash Bulk Bitwise Operations Using Inherent Computation Capability of NAND Flash Memory. In *MICRO*, 2022.
- [157] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, Shuotao Xu, and Arvind. BlueDBM: An Appliance for Big Data Analytics. In *ISCA*, 2015.
- [158] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, and Shuotao Xu. BlueDBM: Distributed Flash Storage for Big Data Analytics. *TOCS*, 2016.
- [159] Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Hosein Bobarshad, Vladimir Alves, and Nader Bagherzadeh. Catalina: In-storage Processing Acceleration for Scalable Big Data Analytics. *Euromicro PDP*, 2019.
- [160] Joo Hwan Lee, Hui Zhang, Veronica Lagrange, Praveen Krishnamoorthy, Xiaodong Zhao, and Yang Seok Ki. SmartSSD: FPGA Accelerated Near-Storage Data Analytics on SSD. *IEEE Computer Architecture Letters*, 2020.
- [161] Mohamadamin Ajdari, Pyeongsu Park, Joonsung Kim, Dongup Kwon, and Jangwoo Kim. CIDR: A Cost-effective In-line Data Reduction System for Terabit-per-second Scale SSD Arrays. In *HPCA*, 2019.
- [162] Benjamin Y Cho, Won Seob Jeong, Doohwan Oh, and Won Woo Ro. XSD: Accelerating Mapreduce by Harnessing the GPU inside an SSD. In *WoNDP*, 2013.
- [163] National Research Council et al. Why Metagenomics? In *The New Science of Metagenomics: Revealing the Secrets of our Microbial Planet*. 2007.

- [164] Centers for Disease Control and Prevention. Using the Latest Technology to Detect Outbreaks and Protect the Public's Health. <https://www.cdc.gov/pulsenet/next-gen-wgs.html>, 2020.
- [165] Diane L Downie, Preetika Rao, Corinne David-Ferdon, Sean Courtney, Justin Lee, Claire Quiner, Pia MacDonald, Keegan Barnes, Shelby S Fisher, Joanne D Andreadis, Jasmine Chaitram, Matthew R Mauldin, Reynolds M Salerno, Jarad Schiffer, and Adi Gundlapalli. 1774. Surveillance for Emerging and Reemerging Pathogens Using Pathogen Agnostic Metagenomic Sequencing in the United States: A Critical Role for Federal Government Agencies. *Open Forum Infectious Diseases*, 2023.
- [166] Centers for Disease Control and Prevention. AMD: Developing Faster Tests. <https://www.cdc.gov/amd/what-we-do/faster-tests.html>, 2019.
- [167] Zachary D. Stephens, Skylar Y. Lee, Faraz Faghri, Roy H. Campbell, Chengxiang Zhai, Miles J. Efron, Ravishankar Iyer, Michael C. Schatz, Saurabh Sinha, and Gene E. Robinson. Big Data: Astronomical or Genomic? *PLOS Biology*, 2015.
- [168] Erika Check Hayden. Genome researchers raise alarm over big data. *Nature*, 2015.
- [169] Robert C. Edgar, Jeff Taylor, Victor Lin, Tomer Altman, Pierre Barbera, Dmitry Meleshko, Dan Lohr, Gherman Novakovsky, Benjamin Buchfink, Basem Al-Shayeb, Jillian F. Banfield, Marcos de la Peña, Anton Korobeynikov, Rayan Chikhi, and Artem Babaian. Petabase-scale sequence alignment catalyses viral discovery. *Nature*, 2022.
- [170] Mantas Sereika, Rasmus Hansen Kirkegaard, Søren Michael Karst, Thomas Yssing Michaelsen, Emil Aarre Sørensen, Rasmus Dam Wollenberg, and Mads Albertsen. Oxford Nanopore R10.4 long-read sequencing enables the generation of near-finished bacterial genomes from pure cultures and metagenomes without short-read or reference polishing. *Nature Methods*, 2022.
- [171] Yunhao Wang, Yue Zhao, Audrey Bollas, Yuru Wang, and Kin Fai Au. Nanopore Sequencing Technology, Bioinformatics and Applications. *Nature Biotechnology*, 2021.
- [172] Leonard Schuele, Hayley Cassidy, Nilay Peker, John W. A. Rossen, and Natacha Couto. Future potential of metagenomics in microbiology laboratories. *Expert Review of Molecular Diagnostics*, 2021.
- [173] Illumina. NovaSeq X Series Specifications. <https://emea.illumina.com/systems/sequencing-platforms/novaseq-x-plus/specifications.html>, 2023.
- [174] Shadi Shokralla, Teresita M. Porter, Joel F. Gibson, Rafal Dobosz, Daniel H. Janzen, Winnie Hallwachs, G. Brian Golding, and Mehrdad Hajibabaei. Massively parallel multiplex DNA sequencing for specimen identification using an Illumina MiSeq platform. *Scientific Reports*, 2015.
- [175] Haowen Zhang, Haoran Li, Chirag Jain, Haoyu Cheng, Kin Fai Au, Heng Li, and Srinivas Aluru. Real-time Mapping of Nanopore Raw Signals. *Bioinformatics*, 2021.
- [176] Can Firtina, Nika Mansouri Ghiasi, Joel Lindegger, Gagandeep Singh, Meryem Banu Cavlak, Haiyu Mao, and Onur Mutlu. RawHash: enabling fast and accurate real-time analysis of raw nanopore signals for large genomes. *Bioinformatics*, 2023.
- [177] Sam Kovaka, Yunfan Fan, Bohan Ni, Winston Timp, and Michael C. Schatz. Targeted Nanopore Sequencing by Real-time Mapping of Raw Electrical Signal with UNCALLED. *Nature Biotechnology*, 2020.
- [178] Onur Mutlu and Can Firtina. Accelerating Genome Analysis via Algorithm-Architecture Co-Design. In *DAC*, 2023.
- [179] Alexander Payne, Nadine Holmes, Thomas Clarke, Rory Munro, Bisrat J. Debebe, and Matthew Loose. Readfish enables targeted nanopore sequencing of gigabase-sized genomes. *Nature Biotechnology*, 2021.
- [180] Yuwei Bao, Jack Wadden, John R. Erb-Downward, Piyush Ranjan, Weichen Zhou, Torrin L. McDonald, Ryan E. Mills, Alan P. Boyle, Robert P. Dickson, David Blaauw, and Joshua D. Welch. SquiggleNet: real-time, direct classification of nanopore signals. *Genome Biology*, 2021.
- [181] Jens-Uwe Ulrich, Ahmad Lutfi, Kilian Rutzen, and Bernhard Y Renard. ReadBouncer: precise and scalable adaptive sampling for nanopore sequencing. *Bioinformatics*, 2022.
- [182] Illumina. NovaSeq 6000 System Specifications. <https://emea.illumina.com/systems/sequencing-platforms/novaseq/specifications.html>, 2020.
- [183] Miten Jain, Hugh E. Olsen, Benedict Paten, and Mark Akeson. The Oxford Nanopore MinION: delivery of nanopore sequencing to the genomics community. *Genome Biology*, 2016.
- [184] Aaron Pomerantz, Nicolás Peñafiel, Alejandro Arteaga, Lucas Bustamante, Frank Pichardo, Luis A Coloma, César L Barrio-Amorós, David Salazar-Valenzuela, and Stefan Prost. Real-time DNA Barcoding in a Rainforest Using Nanopore Sequencing: Opportunities for Rapid Biodiversity Assessments and Local Capacity Building. *GigaScience*, 2018.
- [185] Eric W Sayers, Richa Agarwala, Evan E Bolton, J Rodney Brister, Kathi Canese, Karen Clark, Ryan Connor, Nicolas Fiorini, Kathryn Funk, Timothy Hefferon, J Bradley Holmes, Sunghwan Kim, Avi Kimchi, Paul A Kitts, Stacy Lathrop, Zhiyong Lu, Thomas L Madden, Aron Marchler-Bauer, Lon Phan, Valerie A Schneider, Conrad L Schoch, Kim D Pruitt, and James Ostell. Database resources of the National Center for Biotechnology Information. *Nucleic Acids Research*, 2018.
- [186] AMD. AMD® EPYC® 7742 CPU. <https://www.amd.com/en/products/cpu/amd-epyc-7742>.
- [187] Micron Technology Inc. 4Gb: x4, x8, x16 DDR4 SDRAM Data Sheet, 2016.
- [188] Serial ATA International Organization. SATA revision 3.0 specifications. <https://www.sata-io.org>.
- [189] PCI-SIG. PCI Express Base Specification Revision 4.0, Version 1.0. <https://pcisig.com/specifications>.
- [190] Samsung. PM1735. <https://www.digitec.ch/en/s1/product/samsung-pm1735-3200-gb-pci-express-ssd-15678607>.
- [191] Samsung PM9A3. <https://www.digitec.ch/en/s1/product/samsung-pm9a3-3840-gb-m2-22110-ssd-16404342>.
- [192] Samsung 870 EVO. <https://www.digitec.ch/en/s1/product/samsung-870-evo-4000-gb-25-ssd-14599189>.
- [193] Daehwan Kim, Li Song, Florian P Breitwieser, and Steven L Salzberg. Centrifuge. <http://www.ccb.jhu.edu/software/centrifuge/>, 2020.
- [194] Téo Lemane, Paul Medvedev, Rayan Chikhi, and Pierre Peterlongo. kmtricks: efficient and flexible construction of Bloom filters for large sequencing data collections. *Bioinformatics Advances*, 2022.
- [195] Jarno N Alanko, Jaakko Vuotoniemi, Tommi Mäklin, and Simon J Puglisi. Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes. *Bioinformatics*, 2023.
- [196] Jason Fan, Noor Pratap Singh, Jamshed Khan, Giulio Ermanno Pibiri, and Rob Patro. Fulgor: A Fast and Compact k-mer Index for Large-Scale Matching and Color Queries. In *WABI*, 2023.
- [197] Zhuowen Zou, Hanning Chen, Prathyush Poduval, Yeseong Kim, Mahdi Imani, Elahesh Sadredini, Rosario Cammarota, and Mohsen Imani. BioHD: an efficient genome sequence search platform using hyperdimensional memorization. In *ISCA*, 2022.
- [198] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungrinun. A Modern Primer on Processing in Memory. In *Emerging Computing: From Devices to Systems: Looking Beyond Moore and Von Neumann*. 2022.
- [199] Saugata Ghose, Amirali Boroumand, Jeremie S Kim, Juan Gómez-Luna, and Onur Mutlu. Processing-in-Memory: A Workload-Driven Perspective. *IBM Journal of Research and Development*, 2019.
- [200] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungrinun. Processing Data Where It Makes Sense: Enabling In-Memory Computation. *Microprocessors and Microsystems*, 2019.
- [201] Saugata Ghose, Kevin Hsieh, Amirali Boroumand, Rachata Ausavarungrinun, and Onur Mutlu. Enabling the Adoption of Processing-in-memory: Challenges, Mechanisms, Future Research Directions. *arXiv*, 2018.
- [202] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Ma, Joël Lindegger, Meryem Banu Cavlak, Mohammed Alser, Jisung Park, and Onur Mutlu. MegIS: A High-Performance, Energy-Efficient, and Low-Cost In-Storage Processing System for Metagenomic Analysis. *arXiv*, 2024.
- [203] Marek Kokot, Maciej Długosz, and Sebastian Deorowicz. KMC 3: counting and manipulating k-mer statistics. *Bioinformatics*, 2017.
- [204] Nikola Samardžić, Weikang Qiao, Vaibhav Aggarwal, Mau-Chung Frank Chang, and Jason Cong. Bonsai: High-performance adaptive merge tree sorting. In *ISCA*, 2020.
- [205] Weikang Qiao, Licheng Guo, Zhenman Fang, Mau-Chung Frank Chang, and Jason Cong. TopSort: A High-Performance Two-Phase Sorting Accelerator Optimized on HBM-based FPGAs. In *FCCM*, 2022.
- [206] Soundarya Jayaraman, Bingyi Zhang, and Viktor Prasanna. Hypersort: High-performance Parallel Sorting on HBM-enabled FPGA. In *ICFPT*, 2022.
- [207] Gaëtan Benoit, Pierre Peterlongo, Mahendra Mariadassou, Erwan Drezén, Sophie Schbath, Dominique Lavenier, and Claire Lemaître. Multiple comparative metagenomics using multitest k-mer counting. *PeerJ Computer Science*, 2016.
- [208] Roderick Bovee and Nick Greenfield. Finch: a tool adding dynamic abundance filtering to genomic minhashing. *The Journal of Open Source Software*, 2018.
- [209] Samsung. Samsung SSD 980 PRO. <https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/980pro/>, 2020.
- [210] Shaopeng Liu and David Koslicki. CMash: fast, multi-resolution estimation of k-mer-based Jaccard and containment indices. *Bioinformatics*, 2022.
- [211] Camille Marchet, Christina Boucher, Simon J. Puglisi, Paul Medvedev, Mikael Salson, and Rayan Chikhi. Data structures based on k-mers for querying large collections of sequencing data sets. *Genome Research*, 2021.
- [212] Silvio Weging, Andreas Gogol-Döring, and Ivo Grosse. Taxonomic analysis of metagenomic data with kASA. *Nucleic Acids Research*, 2021.
- [213] Anirban Nag, CN Ramachandra, Rajeev Balasubramanian, Ryan Stutsman, Edouard Giacomini, Hari Kambalasubramanyam, and Pierre-Emmanuel Gaillardon. Gen-Cache: Leveraging In-cache Operators for Efficient Sequence Alignment. In *MICRO*, 2019.
- [214] Daichi Fujiki, Arun Subramaniyan, Tianjun Zhang, Yu Zeng, Reetuparna Das, David Blaauw, and Satish Narayanasamy. Genax: A Genome Sequencing Accelerator. In *ISCA*, 2018.
- [215] Myungsu Kim, Jaehoon Lee, Sungjin Lee, Jisung Park, Youngsun Song, and Jihong Kim. Improving Performance and Lifetime of Large-page NAND Storages Using Erase-free Subpage Programming. In *DAC*, 2017.
- [216] Atsuo Kawaguchi, Shingo Nishioka, and Hiroshi Motoda. A flash-memory based file system. In *ATC*, 1995.
- [217] Micron. Product Flyer: Micron 3D NAND Flash Memory. https://www.micron.com/-/media/client/global/documents/products/product-flyer/3d_nand_flyer.pdf?la=en, 2016.
- [218] Synopsys, Inc. Design Compiler. <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-graphical.html>.
- [219] United Microelectronics Corporation. UMK65LSCLLMVBBL_A - UMC 65 nm Low-K 1.2V/1.0V Low Leakage LVT Tapless Standard Cell Library, version A02, 2008.
- [220] Cadence Design Systems, Inc. Innovus Implementation System. https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/soc-implementation-and-floorplanning/innovus-implementation-system.html.
- [221] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A Fast and Extensible DRAM Simulator. *CAL*, 2015.
- [222] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator Source Code. <https://github.com/CMU-SAFARI/ramulator>.
- [223] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim: A Framework for Enabling Realistic Studies of Modern Multi-queue SSD Devices. In *FAST*, 2018.
- [224] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim Source Code. <https://github.com/CMU-SAFARI/MQSim>.
- [225] Samsung. LPDDR4. <https://semiconductor.samsung.com/dram/lpddr/lpddr4/>.
- [226] Saugata Ghose, Tianshi Li, Nastaran Hajinazar, Damla Senol Cali, and Onur Mutlu. Demystifying Complex Workload-DRAM Interactions: An Experimental Study.

- POMACS, 2019.
- [227] Advanced Micro Devices. AMD® μ Prof. <https://developer.amd.com/amd-uprof/>, 2021.
 - [228] Alexander Sczyrba, Peter Hofmann, Peter Belmann, David Koslicki, Stefan Janssen, Johannes Dröge, Ivan Gregor, Stephan Majda, Jessica Fiedler, Eik Dahms, Andreas Bremges, Adrian Fritz, Ruben Garrido-Oter, Tue Sparholt Jørgensen, Nicole Shapiro, Philip D. Blood, Alexey Gurevich, Yang Bai, Dmitriy Turaev, Matthew Z. DeMaere, Rayan Chikhi, Niranjan Nagarajan, Christopher Quince, Fernando Meyer, Monika Balvočiūtė, Lars Hestbjerg Hansen, Søren J. Sørensen, Burton K. H. Chia, Bertrand Denis, Jeff L. Froula, Zhong Wang, Robert Egan, Dongwan Don Kang, Jeffrey J. Cook, Charles Dettel, Michael Beckstette, Claire Lemaitre, Pierre Peterlongo, Guillaume Rizk, Dominique Lavenier, Yu-Wei Wu, Steven W. Singer, Chirag Jain, Marc Strous, Heiner Klingenberg, Peter Meinicke, Michael D. Barton, Thomas Lingner, Hsin-Hung Lin, Yu-Chieh Liao, Genivaldo Gueiros Z. Silva, Daniel A. Cuevas, Robert A. Edwards, Surya Saha, Vitor C. Piro, Bernhard Y. Renard, Mihai Pop, Hans-Peter Klenk, Markus Göker, Nikos C. Kyrpides, Tanja Woyke, Julia A. Vorholt, Paul Schulze-Lefert, Edward M. Rubin, Aaron E. Darling, Thomas Rattei, and Alice C. McHardy. Critical Assessment of Metagenome Interpretation—A Benchmark of Metagenomics Software. *Nature Methods*, 2017.
 - [229] Oxford Nanopore Technologies. MinION Mk1B IT Requirements. https://community.nanoporetech.com/requirements_documents/minion-it-reqs.pdf, 2021.
 - [230] Damla Senol Cali, Jeremie S Kim, Saugata Ghose, Can Alkan, and Onur Mutlu. Nanopore Sequencing Technology and Tools for Genome Assembly: Computational Analysis of the Current State, Bottlenecks and Future Directions. *Briefings in Bioinformatics*, 2018.
 - [231] Aaron Stillmaker and Bevan Baas. Scaling Equations for the Accurate Prediction of CMOS Device Performance from 180 Nm to 7 Nm. *Integration*, 2017.
 - [232] Xiaoquan Su, Jian Xu, and Kang Ning. Parallel-meta: efficient metagenomic data analysis based on high-performance computation. *BMC Systems Biology*, 2012.
 - [233] Xiaoquan Wang, Xuetao Wang, Gongchao Jing, and Kang Ning. GPU-Meta-Storms: computing the structure similarities among massive amount of microbial community samples using GPU. *Bioinformatics*, 2013.
 - [234] Masahiro Yano, Hiroshi Mori, Yutaka Akiyama, Takuji Yamada, and Ken Kurokawa. Clast: Cuda implemented large-scale alignment search tool. *BMC Bioinformatics*, 2014.
 - [235] Antonio Saavedra, Hans Lehnert, Cecilia Hernández, Gonzalo Carvajal, and Miguel Figueroa. Mining discriminative k-mers in DNA sequences using sketches and hardware acceleration. *IEEE Access*, 2020.
 - [236] Tianqi Zhang, Antonio González, Niema Moshiri, Rob Knight, and Tajana Rosing. GenoMix: Accelerated Simultaneous Analysis of Human Genomics, Microbiome Metagenomics, and Viral Sequences. In *BioCAS*, 2023.
 - [237] Gustavo Henrique Cervi, Cecilia Dias Flores, and Claudia Elizabeth Thompson. Metagenomic Analysis: A Pathway Toward Efficiency Using High-Performance Computing. In *ICICT*, 2022.
 - [238] Tim Dunn, Harisankar Sadasivan, Jack Wadden, Kush Goliya, Kuan-Yu Chen, David Blaauw, Reetuparna Das, and Satish Narayanasamy. SquiggleFilter: An Accelerator for Portable Virus Detection. In *MICRO*, 2021.
 - [239] Po Jui Shih, Hassaan Saadat, Sri Parameswaran, and Hasindu Gamaarachchi. Efficient real-time selective genome sequencing on resource-constrained devices. *GigaScience*, 2023.
 - [240] Omar Ahmed, Massimiliano Rossi, Sam Kovaka, Michael C. Schatz, Travis Gagie, Christina Boucher, and Ben Langmead. Pan-genomic matching statistics for targeted nanopore sequencing. *iScience*, 2021.
 - [241] Wenqin Huangfu, Shuangchen Li, Xing Hu, and Yuan Xie. RADAR: A 3D-ReRAM based DNA Alignment Accelerator Architecture. In *DAC*, 2018.
 - [242] S Karen Khatamifard, Zamshed Chowdhury, Nakul Pande, Meisam Razaviyayn, Chris H Kim, and Ulya R Karpuzcu. GENVoM: Read Mapping Near Non-Volatile Memory. *TCBB*, 2021.
 - [243] Xue-Qi Li, Guang-Ming Tan, and Ning-Hui Sun. PIM-Align: A Processing-in-Memory Architecture for FM-Index Search Algorithm. *Journal of Computer Science and Technology*, 2021.
 - [244] Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. Aligns: A Processing-in-memory Accelerator for DNA Short Read Alignment Leveraging SOT-MRAM. In *DAC*, 2019.
 - [245] Farzaneh Zokaei, Hamid R Zareandi, and Lei Jiang. Aligner: A Process-in-memory Architecture for Short Read Alignment in ReRAMs. *IEEE Computer Architecture Letters*, 2018.
 - [246] Advait Madhavan, Timothy Sherwood, and Dmitri Strukov. Race Logic: A Hardware Acceleration for Dynamic Programming Algorithms. *SIGARCH Computer Architecture News*, 2014.
 - [247] Haoyu Cheng, Yong Zhang, and Yun Xu. Bitmapper2: A GPU-accelerated All-mapper Based on The Sparse Q-gram Index. *TCBB*, 2018.
 - [248] Ernst Joachim Houtgast, Vlad-Mihai Sima, Koen Bertels, and Zaid Al-Ars. Hardware Acceleration of BWA-MEM Genomic Short Read Mapping for Longer Read Lengths. *Computational Biology and Chemistry*, 2018.
 - [249] Ernst Joachim Houtgast, Vlad-Mihai Sima, Koen Bertels, and Zaid Al-Ars. An Efficient GPU-accelerated Implementation of Genomic Short Read Mapping with BWA-MEM. *SIGARCH Computer Architecture News*, 2017.
 - [250] Alberto Zeni, Giulia Guidi, Marquita Ellis, Nan Ding, Marco D Santambrogio, Steven Hofmeyr, Aydın Buluç, Leonid Oliker, and Katherine Yelick. Logan: High-performance GPU-based X-drop Long-read Alignment. In *IPDPS*, 2020.
 - [251] Nauman Ahmed, Jonathan Lévy, Shanshan Ren, Hamid Mushtaq, Koen Bertels, and Zaid Al-Ars. GASAL2: A GPU Accelerated Sequence Alignment Library for High-Throughput NGS Data. *BMC Bioinformatics*, 2019.
 - [252] Takahiro Nishimura, Jacir I Bordim, Yasuaki Ito, and Koji Nakano. Accelerating the Smith-Waterman Algorithm Using Bitwise Parallel Bulk Computation Technique on GPU. In *IPDPSW*, 2017.
 - [253] Edans Flavius de Oliveira Sandes, Guillermo Miranda, Xavier Martorell, Eduard Ayguade, George Teodoro, and Alba Cristina Magalhães Melo. CUDAlign 4.0: Incremental Speculative Traceback for Exact Chromosome-wide Alignment in GPU Clusters. *TPDS*, 2016.
 - [254] Yongchao Liu and Bertil Schmidt. GSWABE: Faster GPU-accelerated Sequence Alignment with Optimal Alignment Retrieval for Short DNA Sequences. *Concurrency and Computation: Practice and Experience*, 2015.
 - [255] Yongchao Liu, Adrianto Wirawan, and Bertil Schmidt. CUDASW++ 3.0: Accelerating Smith-Waterman Protein Database Search by Coupling CPU and GPU SIMD Instructions. *BMC Bioinformatics*, 2013.
 - [256] Yongchao Liu, Douglas L Maskell, and Bertil Schmidt. CUDASW++: Optimizing Smith-Waterman Sequence Database Searches for CUDA-enabled Graphics Processing Units. *BMC Research Notes*, 2009.
 - [257] Yongchao Liu, Bertil Schmidt, and Douglas L Maskell. CUDASW++ 2.0: Enhanced Smith-Waterman Protein Database Search on CUDA-enabled GPUs Based on SIMD and Virtualized SIMD Abstractions. *BMC Research Notes*, 2010.
 - [258] Richard Wilton, Tamas Budavari, Ben Langmead, Sarah J Wheelan, Steven L Salzberg, and Alexander S Szalay. Arioc: High-throughput Read Alignment with GPU-accelerated Exploration of The Seed-and-extend Search Space. *PeerJ*, 2015.
 - [259] Amit Goyal, Hyuk Jung Kwon, Kichan Lee, Reena Garg, Seon Young Yun, Yoon Hee Kim, Sunghoon Lee, and Min Seob Lee. Ultra-fast Next Generation Human Genome Sequencing Data Processing Using DRAGENTM Bio-IT Processor for Precision Medicine. *Open Journal of Genetics*, 2017.
 - [260] Yu-Ting Chen, Jason Cong, Zhenman Fang, Jie Lei, and Peng Wei. When Spark Meets FPGAs: A Case Study for Next-Generation DNA Sequencing Acceleration. In *HotCloud*, 2016.
 - [261] Peng Chen, Chao Wang, Xi Li, and Xuehai Zhou. Accelerating the Next Generation Long Read Mapping with the FPGA-based System. *TCBB*, 2014.
 - [262] Yen-Lung Chen, Bo-Yi Chang, Chia-Hsiang Yang, and Tzi-Dar Chiueh. A High-Throughput FPGA Accelerator for Short-Read Mapping of the Whole Human Genome. *TPDS*, 2021.
 - [263] Daichi Fujiki, Shunhao Wu, Nathan Ozog, Kush Goliya, David Blaauw, Satish Narayanasamy, and Reetuparna Das. SeedEx: A Genome Sequencing Accelerator for Optimal Alignments in Subminimal Space. In *MICRO*, 2020.
 - [264] Subho Sankar Banerjee, Mohamed El-Hadedy, Jong Bin Lim, Zbigniew T Kalbarczyk, Deming Chen, Steven S Lumetta, and Ravishankar K Iyer. ASAP: Accelerated Short-read Alignment on Programmable Hardware. *TC*, 2019.
 - [265] Xia Fei, Zou Dan, Lu Lina, Man Xin, and Zhang Chunlei. FPGASW: Accelerating Large-scale Smith-Waterman Sequence Alignment Application with Backtracking on FPGA Linear Systolic Array. *Interdisciplinary Sciences: Computational Life Sciences*, 2018.
 - [266] Hasitha Muthumala Waidyasooriya and Masanori Hariyama. Hardware-acceleration of Short-read Alignment Based on the Burrows-Wheeler Transform. *TPDS*, 2015.
 - [267] Yu-Ting Chen, Jason Cong, Jie Lei, and Peng Wei. A Novel High-throughput Acceleration Engine for Read Alignment. In *FCCM*, 2015.
 - [268] Enzo Rucci, Carlos Garcia, Guillermo Botella, Armando De Giusti, Marcelo Naiouf, and Manuel Prieto-Matias. SWIFOLD: Smith-Waterman Implementation on FPGA with OpenCL for Long DNA Sequences. *BMC Systems Biology*, 2018.
 - [269] Abbas Haghi, Santiago Marco-Sola, Lluç Alvarez, Dionysios Diamantopoulos, Christoph Hagleitner, and Miquel Moreto. An FPGA Accelerator of the Wavefront Algorithm for Genomics Pairwise Alignment. *FPL*, 2021.
 - [270] Luyi Li, Jun Lin, and Zhongfeng Wang. PipeBSW: A Two-Stage Pipeline Structure for Banded Smith-Waterman Algorithm on FPGA. *ISVLSI*, 2021.
 - [271] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Jae W Lee, David Bruns-Smith, Brendan Sweeney, Krste Asanovic, Young H Oh, and Lisa Wu Wills. Accelerating Genomic Data Analytics With Composable Hardware Acceleration Framework. *IEEE Micro*, 2021.
 - [272] Lisa Wu, David Bruns-Smith, Frank A. Nothaft, Qijing Huang, Sagar Karandikar, Johnny Le, Andrew Lin, Howard Mao, Brendan Sweeney, Krste Asanovic, David A. Patterson, and Anthony D. Joseph. FPGA Accelerated Indel Realignment in the Cloud. In *HPCA*, 2019.
 - [273] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. Benchmarking learned indexes. *Proc. VLDB Endow.*, 2020.
 - [274] Arun Subramaniam, Jack Wadden, Kush Goliya, Nathan Ozog, Xiao Wu, Satish Narayanasamy, David Blaauw, and Reetuparna Das. Accelerated seeding for genome sequence alignment with enumerated radix trees. In *ISCA*, 2021.
 - [275] Shengwen Liang, Ying Wang, Cheng Liu, Huawei Li, and Xiaowei Li. InS-DLA: An In-SSD deep learning accelerator for near-data processing. In *FPL*, 2019.
 - [276] Lingxi Wu, Minxuan Zhou, Weihong Xu, Ashish Venkat, Tajana Rosing, and Kevin Skadron. Abakus: Accelerating k-mer Counting With Storage Technology. *TACO*, 2023.